

SmolVLA: A vision-language-action model for affordable and efficient robotics

Mustafa Shukor*^S Dana Aubakirova*[🤖] Francesco Capuano*[🤖]

Pepijn Kooijmans[🤖] Steven Palma[🤖] Adil Zouitine[🤖] Michel Aractingi[🤖] Caroline Pascal[🤖] Martino Russi[🤖] Andres Marafioti[🤖] Simon Alibert[🤖] Matthieu Cord^{S v} Thomas Wolf[🤖] Remi Cadene*[🤖]

[🤖] Hugging Face, ^S Sorbonne University, ^v valeo.ai, ^{🇫🇷} École Normale Supérieure Paris-Saclay * Core team

Abstract



Vision-language models (VLMs) pretrained on large-scale multimodal datasets encode rich visual and linguistic knowledge, making them a strong foundation for robotics. Rather than training robotic policies from scratch, recent approaches adapt VLMs into vision-language-action (VLA) models that enable natural language-driven perception and control. However, existing VLAs are typically massive—often with billions of parameters—leading to high training costs and limited real-world deployability. Moreover, they rely on academic and industrial datasets, overlooking the growing availability of community-collected data from affordable robotic platforms. In this work, we present SmolVLA, a small, efficient, and community-driven VLA that drastically reduces both training and inference costs, while retaining competitive performance. SmolVLA is designed to be trained on a single GPU and deployed on consumer-grade GPUs or even CPUs. To further improve responsiveness, we introduce an asynchronous inference stack decoupling perception and action prediction from action execution, allowing higher control rates with chunked action generation. Despite its compact size, SmolVLA achieves performance comparable to VLAs that are 10× larger. We evaluate SmolVLA on a range of both simulated as well as real-world robotic benchmarks and release all code, pretrained models, and training data.

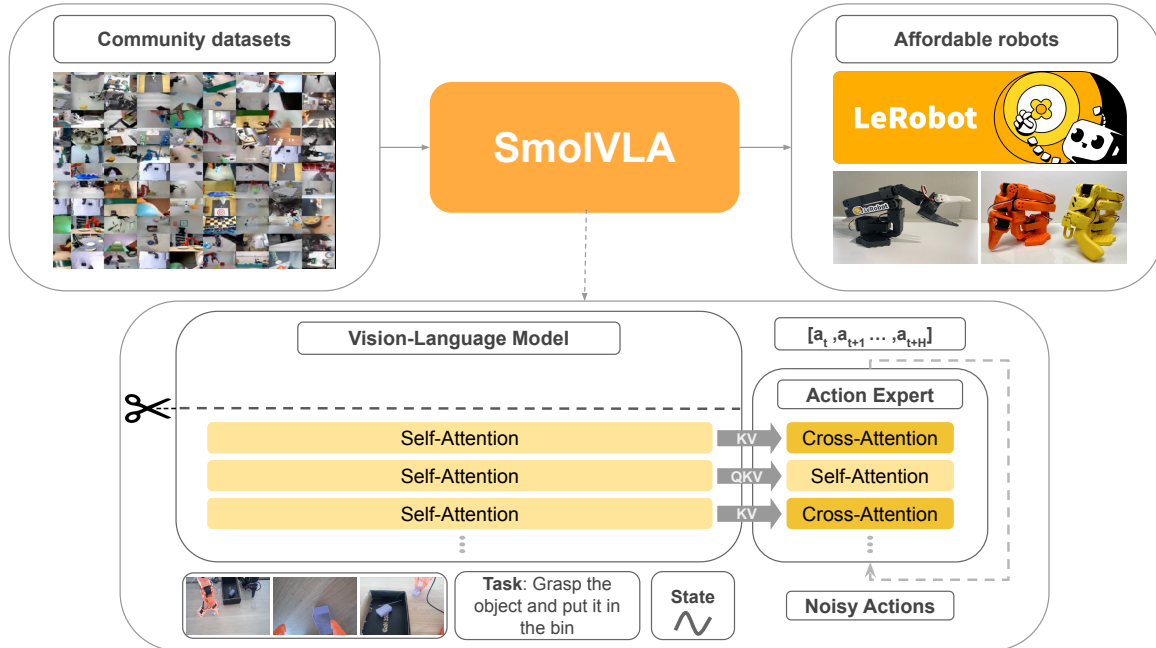


Figure 1 | SmolVLA. SmolVLA consists of a compact pretrained vision-language model, discarding the last $L - N$ layers (scissors icon). The remaining layers embed three inputs: (i) language instruction, (ii) RGB image(s), and (iii) robot sensorimotor state. Their merged tokens feed an Action Expert of alternating cross-attention (gold) and self-attention (light yellow) blocks, trained with flow matching to output n low-level actions chunk $a_t, \dots, a_{t+n}$. SmolVLA is pretrained on public community datasets and evaluated on low-cost robots.

1 Introduction

In recent years, the field has shifted towards the development of *foundation* models, generalist models capable of performing a wide range of tasks. A prominent example of this trend are large language models (LLMs), which have demonstrated performance comparable to the average human in understanding and generating natural language, reasoning over complex topics, and anchoring in knowledge (Brown et al., 2020; Achiam et al., 2023; Dubey et al., 2024; Team et al., 2023; Jiang et al., 2023). The success of text-based models has thus been extended to other modalities, sparking interest towards multi-modal vision-language (VLMs) (Alayrac et al., 2022; Chen et al., 2023; Huang et al., 2023; Liu et al., 2023b; Chen et al., 2024; Shukor et al., 2023b) and audio-language models (ALMs) (Défossez et al., 2024; Das et al., 2024; Borsos et al., 2023). While complementary in terms of modalities, such advances in developing multi-modal foundation models stem from (i) the adoption of scalable architectures, such as the Transformer (Vaswani, 2017) and (ii) internet-scale training datasets.

Despite their remarkable achievements in the digital world, real-world application of foundation models—particularly in robotics—remains limited. In particular, robotic policies (Zhao et al., 2023; Chi et al., 2024; Lee et al., 2024; Hansen et al., 2022) still face challenges in generalizing across object types, positions, environments, and tasks (Xie et al., 2024; Ebert et al., 2021). Robots should be able to adapt to new surroundings and new objects, which requires robust skills and common sense understanding of the world. Yet, the progress in this direction seems to be often limited by the availability of high-quality and diverse data.

To address this limitation, a growing body of work has begun exploring robotics foundation models in the form of vision-language-action (VLA) models (Team et al., 2024; O’Neill et al., 2024; Brohan et al., 2023; Kim et al., 2024; Black et al., 2024; Bjorck et al., 2025; Li et al., 2024; Huang et al., 2024). VLAs are designed to incorporate abstract reasoning, world knowledge, and decision-making skills embedded in pretrained large language and vision-language models. These models take multimodal inputs—such as visual observations and natural language instructions—and predict the corresponding robotic actions. Early results suggest promising gains in generalization capabilities (Black et al., 2024; Brohan et al., 2023).

VLA models remain in an early stage of development and are not yet as mature or widely adopted as LLMs and VLMs. Much of the impactful VLA progress remains proprietary, with many models sharing only weights while withholding full training details and essential methodological components. While effective in tackling academic benchmarks, we argue achieving human-level capabilities in robotics requires a stronger commitment to open-source efforts. In particular, transparent, reproducible open-source models and training recipes are crucial for accelerating progress and fostering broader participation within the robotics research community. We advocate for the development of affordable, efficient models that are accessible to a broader community. While encouraging efforts like OpenVLA (Kim et al., 2024) and RT-2-X (O’Neill et al., 2024) demonstrate the feasibility of open VLA systems, they remain large, resource-intensive, and dependent on costly robotic platforms, hindering accessibility.

In this work, we introduce SmolVLA, an open-source initiative featuring a compact yet capable VLA model, released alongside reproducible and efficient training and inference recipes. Our contributions are as follows:

- **Lightweight architecture.** We present SmolVLA, a compact and efficient vision-language agent optimized for training on consumer-grade GPUs and deployment on CPUs. Key design choices include: (i) skipping layers in the VLM, (ii) using a minimal number of visual tokens (iii) leveraging small pretrained VLMs, and (iv) interleaving self-attention layers with lighter cross-attention layers.
- **Pretraining on community-driven datasets.** SmolVLA is trained end-to-end on fewer than 30k episodes drawn entirely from publicly available, community-contributed datasets, demonstrating strong performance while using an order of magnitude less data than prior art.
- **Asynchronous inference.** We introduce an optimized asynchronous inference stack that decouples action execution from observation processing and action prediction, reducing latency and enabling fast, resource-efficient inference.

We evaluate SmolVLA on both simulated environments and real-world settings on multiple tasks. Interestingly, although significantly smaller, SmolVLA matches or surpasses the performance of much larger VLA models.

2 Related work

Vision-language models (VLMs). VLMs are designed to process both visual and textual modalities—most commonly by taking both images and text as input and generating text conditioned on the visual context. Recent advances in

VLMs have been driven by the success of LLMs, with many approaches building upon pretrained LLMs and adopting similar training paradigms. Typically, VLMs (Alayrac et al., 2022; Laurençon et al., 2024; Lin et al., 2023a) are constructed by integrating a pretrained vision encoder (Radford et al., 2021; Zhai et al., 2023; Fini et al., 2024) with a pretrained LLM (AI@Meta, 2024; Jiang et al., 2023; Wang et al., 2024). Training then proceeds in multiple multimodal stages, beginning with large-scale pretraining on image-caption datasets (Schuhmann et al., 2022; Byeon et al., 2022) and interleaved vision-language corpora (Laurençon et al., 2023; Zhu et al., 2023), all followed by a supervised fine-tuning stage on instruction-tuning datasets (Liu et al., 2023b; Tong et al., 2024; Laurençon et al., 2024). Other works have shown the benefits of not relying on pretrained vision encoders (Bavishi et al., 2023; Shukor et al., 2025; Diao et al., 2025, 2024), while other works aim at developing more unified architectures representing both images and text as discrete tokens, enabling a single model to process multimodal sequences of tokens (Wang et al., 2022; Shukor et al., 2023b; Team, 2024; Lin et al., 2024). Efficiency has also become a central focus in VLM research. Several works aim to reduce training costs by using smaller, more diverse datasets (Liu et al., 2023b; Dai et al., 2023; Bai et al., 2025; Zhu et al., 2024; Tong et al., 2024), training smaller-scale models (Marafioti et al., 2025; Korrapati, 2024; Yao et al., 2024), or by adapting pretrained unimodal models by tuning only a small subset of parameters (Shukor et al., 2023a; Vallaes et al., 2024; Mañas et al., 2023; Koh et al., 2023; Tsimpoukelli et al., 2021; Li et al., 2023). While the majority of VLM research focuses on image and text modalities, recent work has demonstrated that similar techniques can be extended to integrate additional modalities, such as video and audio (Wang et al., 2025; Liu et al., 2024; Zhang et al., 2025; Kong et al., 2024).

Vision-language-action models (VLAs). A growing area of interest in robotics research is the development of generalist policies—models capable of performing a wide range of tasks, generalizing across different environments and robot embodiments. A prominent strategy in this direction leverages VLAs, models capable of processing *(i)* task instructions given in natural language, *(ii)* visual observations (e.g., images coming from camera streams), and *(iii)* proprioceptive inputs to output control actions. Early efforts such as Octo (Team et al., 2024) and RT-1 (O’Neill et al., 2024) trained transformer-based models from scratch on large-scale robotic demonstration datasets. To improve both performance and generalization, RT-2 (Brohan et al., 2023) leveraged pretrained vision-language models (VLMs), further training them on robotics-specific data. In an effort to promote openness and reproducibility, OpenVLA (Kim et al., 2024) released a 7B-parameter VLA trained on publicly available data to generate discrete action tokens. As action tokenization poses limitations for continuous control, π_0 (Black et al., 2024) and DexVLA (Wen et al., 2025) proposed using diffusion-based decoders for continuous action generation. In this, both Black et al. (2024); Wen et al. (2025) propose adapting a pretrained VLM, RDT-1B, introducing a large diffusion component—termed *action expert*—trained directly on robotic demonstrations. Recently, Pertsch et al. (2025) proposed a fully autoregressive approach using a novel action tokenizer, improving over traditional binning methods but still suffering from slow (autoregressive) inference. In an effort to improve VLAs’ efficiency, TinyVLA (Wen et al., 2024) trained a lightweight sub-1B model from scratch on multimodal data and then fine-tuned it on robotics datasets, although the absence of large-scale pretraining on robotics data hinders wider generalization capabilities. SmolVLA shares similar goals with most of these efforts, aiming to develop and release open-source models that are both performant and highly efficient in terms of training and inference.

3 SmolVLA: small, efficient and capable

Overview. SmolVLA is a lightweight VLA composed of a compact pretrained VLM, and an action expert trained with flow matching. Given multiple images and a language instruction describing the task, the model outputs a chunk of actions. It is first pretrained with imitation learning on community-collected datasets, then evaluated across both real-world and simulated environments. The pretraining data is designed to span a diverse set of tasks and behaviors, allowing the model to learn generalizable physical skills that transfer across settings. At inference time, we introduce an asynchronous execution stack that decouples action execution from perception and prediction, enabling faster and more responsive control.

3.1 Model architecture

SmolVLA is composed of two main components: *(i)* a pretrained VLM tasked with perception and *(ii)* an action expert, trained to act. The two components are interconnected, as the VLM processes state inputs to generate features that condition the action expert, which produces actions in turn altering the state fed to the VLM. Specifically, the VLM processes sensorimotor states, including images from multiple RGB cameras, and a language instruction describing the task. In turn, the VLM outputs features directly fed to the action expert, which outputs the final

continuous actions.

Vision-language model (VLM). We leverage a pretrained VLM as the main backbone for perceiving the robot’s environment. VLMs, pretrained on diverse multimodal data, capture rich world knowledge. To guarantee efficiency and accessibility, we choose SmolVLM-2 (Marafioti et al., 2025), an efficient model optimized for multi-image and video inputs. SmolVLM-2 relies on SigLIP (Zhai et al., 2023) to encode visual features for SmolLM2 language decoder (Allal et al., 2025). In SmolVLA, the VLM component processes image sequences using the vision encoder, which reduces token count through a token-shuffling technique for efficiency. The language instruction is tokenized into text tokens. Sensorimotor states are projected into a single token via a linear layer to match the language model’s token dimension. Lastly, visual, language, and state tokens are concatenated and passed to the language decoder. The resulting obtained via the decoder layers are then used to condition the action expert.

State, action, and feature projectors. We use linear projection layers in various points inside of SmolVLA. In particular, we use linear projection layers to (i) project the states to match the VLM dimension (ii) project the actions to match the action expert dimensions and (iii) to adapt the VLM features to align with the action expert’s dimension.

Visual tokens reduction. While proven critical for VLM performance, high-resolution images increase inference costs. To guarantee efficiency, SmolVLM-2 is trained with image tiling (Lin et al., 2023b), a popular technique that involves processing multiple crops of the same image, in addition to the global image. However, to obtain a faster inference time, we do not use tiling. We use the global image only, in addition to a pixel shuffle operation, limiting the visual tokens to 64 per frame.

Faster inference through layer skipping. To get faster inference time, we skip computations in the VLM. Prior work (Shukor and Cord, 2024; Tang et al., 2023) demonstrated the possibility of skipping layers in pretrained models without incurring in significant performance degradation. Recently, (El-Nouby et al., 2024; Bolya et al., 2025; Rajasegaran et al., 2025) have shown that the best features for downstream tasks are not necessarily obtained from the last layer of a VLM. Consequently, rather than using the last layer features, our action expert has access to all features up to a specified layer N . In practice, we find setting N to half the total layers ($N = L/2$) offers a good tradeoff between speed and performance, effectively halving the LLM and action expert’s computational cost.

Flow matching action expert. The action expert $\mathbf{v}_\theta$ is trained to predict an action chunk $\mathbf{A}_t = (a_t, \dots, a_{t+n})$ from VLM features. In keeping with prior work, our implementation of $\mathbf{v}_\theta$ relies on the transformer architecture (Vaswani, 2017). Differently from prior VLA architectures, we interleave cross and self-attention layers, thus using a conditional Flow Matching Transformer (Esser et al., 2024; Liu, 2022; Lipman et al., 2022) as $\mathbf{v}_\theta$. The action expert is trained using the objective defined by:

$$\mathcal{L}^\tau(\theta) = \mathbb{E}_{p(\mathbf{A}_t|\mathbf{o}_t), q(\mathbf{A}_t^\tau|\mathbf{A}_t)} \left[\left\| \mathbf{v}_\theta(\mathbf{A}_t^\tau, \mathbf{o}_t) - \mathbf{u}(\mathbf{A}_t^\tau | \mathbf{A}_t) \right\|^2 \right],$$

where $\mathbf{o}_t$ represents the VLM features extracted from an observation o_t at the N -th VLM layer, and $\mathbf{A}_t^\tau = \tau \mathbf{A}_t + (1-\tau)\epsilon$, with $\epsilon \sim \mathcal{N}(0, \mathbf{I})$. In particular, $\mathbf{v}_\theta$ is trained to output the vector field $\mathbf{u}(\mathbf{A}_t^\tau | \mathbf{A}_t) = \epsilon - \mathbf{A}_t$ from the VLM features and the noisy actions $\mathbf{A}_t^\tau$. In keeping with Black et al. (2024), we sample τ from a Beta distribution. To improve on inference’s efficiency, we use a reduced hidden size of $0.75 \times d$ for $\mathbf{v}_\theta$, where d is the VLM’s hidden dimension.

Interleaved cross and causal self-attention layers. The action expert $\mathbf{v}_\theta$ generates action chunks conditioned on the VLM features, with the interaction between the VLM and action expert in SmolVLA being facilitated by the attention mechanisms. Unlike prior works relying exclusively on self-attention (SA) (Black et al., 2024) or cross-attention (CA) (Bjorck et al., 2025), we employ an interleaved approach, where each block contains either a CA or a SA layer. This design choice also differs from standard VLM architectures, where each decoder block typically includes both SA and CA layers (Laurençon et al., 2023; Alayrac et al., 2022; Chen et al., 2022). Within the action expert’s forward pass, the interaction between actions and VLM features takes place via attention, projecting tokens into query, keys and values (Vaswani, 2017). In our setup, CA layers cross-attend the VLM’s keys and values, while SA layers allow the action tokens in $\mathbf{v}_\theta$ to attend to each other. We employ a causal attention mask for the SA layers, ensuring that each action token can only attend to past tokens within the chunk, preventing future action dependencies. Empirically, we find that interleaving CA and SA layers provides higher success rates and faster inference time. In particular, we

find self-attention to contribute to smoother action chunks $\mathbf{A}$, something particularly evident when evaluating on real robots.

3.2 Pretraining data collected by the community

In robotics, the data available for large-scale pretraining remains orders of magnitude smaller than what has driven recent breakthroughs in vision and language. For instance, while natural language foundation models can benefit from the unique text-based interface and massive scale of internet-data, the integration and scaling up of robotics datasets appears complex, due to the (i) differences across datasets and (ii) reliance on teleoperation by human experts for data collection. Additionally, the high heterogeneity of robot morphologies, sensors, actuation modes, control frequencies, and data formats results in "data islands" Bjorck et al. (2025)—scattered robotics datasets whose integration proves challenging.

In this context, the advent of low-end robotic platforms and standardized robotics libraries directly mitigates this data heterogeneity, providing a unique entry point to robotics for practitioners. Further, open-source data contributions collected by individual practitioners enable the larger robotics community with *community datasets*, datasets collected in diverse real-world settings—from academic labs to homes—as part of a larger effort to decentralize and scale robot learning with open-source. Unlike academic datasets following standardized protocols, community datasets naturally span varied robot embodiments, control schemes, camera perspectives, and tasks. Further, community datasets reflect real-world complexity through noisy demonstrations, heterogeneous environments, and diverse object interactions, providing valuable as pre-training data. In this work, we selected a subset of 481 community datasets obtained from Hugging Face, filtered according to embodiment type, episode count, overall data quality, and frame coverage (Table 1).

Task annotation with VLM. Relying on community-contributed datasets entails standardization challenges. In particular, we observed substantial noise in task annotations—natural language descriptions of the robot’s intended behavior for a given dataset. Critically, various datasets included ambiguous placeholders such as `task desc`, overly vague commands such as `Hold` or `Up`, or lacked instructions entirely. To improve on the annotation quality, we used an off-the-shelf VLM (Qwen2.5-VL-3B-Instruct) to auto-generate concise task descriptions. For each dataset, we sampled representative frames and provided them alongside the original instruction. The model was prompted to produce a short, action-oriented sentence summarizing the behavior. The full prompt is available in Appendix A.1.

# datasets	# episodes	# frames
481	22.9K	10.6M

Table 1 | Community datasets statistics. At ~ 10 M episodes, our pretraining set stands at least one order of magnitude smaller than other state-of-the-art. For completeness, we report the list of community-datasets used in Appendix A.1.

Camera viewpoint normalization. Another challenge arising from using community datasets consists in the high variability in the camera naming conventions used. For instance, datasets refer `images.laptop` may refer to a top, side, or wrist-mounted view depending on the particular. We found this inconsistency detrimental during pretraining, and that a consistent camera ordering is rather beneficial for training in this data regime. To address this standardization challenge, we manually mapped each camera to a standardized view type—prioritizing top, wrist, and side perspectives—and renamed them as `OBS_IMAGE_1`, `OBS_IMAGE_2`, and `OBS_IMAGE_3`, respectively. For datasets with additional views, the order was preserved, but unused views were dropped during training. Future efforts may automate this process using VLMs, or propose/adapt standardized data collection guidelines.

3.3 Asynchronous inference

Modern visuomotor policies (Zhao et al., 2023; Chi et al., 2023; Black et al., 2024) output *action chunks*—sequences $\pi(o_t) = \mathbf{A}_t$ with $\mathbf{A}_t = (a_t, a_{t+1}, \dots, a_{t+n})$ being a sequence of n (much greater than 1) low-level commands enqueued in an action queue, originating from an environment observation, o_t . Typically, the robot executes the entire action chunk $\mathbf{A}_t$, before a new observation o_{t+n} is passed to the policy π to predict the next chunk. This results in open-loop inference in between observations captured every n timesteps. Works including Zhao et al. (2023); Chi et al. (2023) adopt a different strategy whereby the robot controller interleaves chunk prediction $\mathbf{A}_t \leftarrow \pi(o_t)$ and chunk consumption $a_t \leftarrow \text{POPFRONT}(\mathbf{A}_t)$, computing a new chunk of actions at every timestep t and aggregating the predicted chunks on overlapping sections. While adaptive—every observation at every timestep o_t is processed—such approaches rely on running inference continuously, which can be prohibitive in resource-constrained scenarios, such as edge deployments.

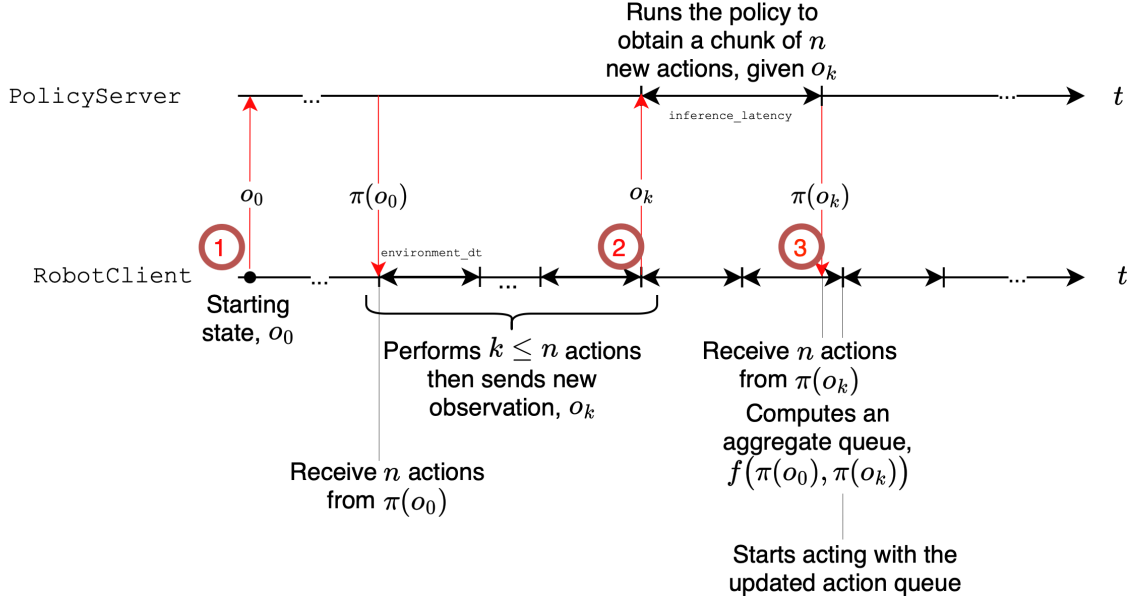


Figure 2 | Asynchronous inference. Illustration of the asynchronous inference stack. Note that the policy can be run on a remote server, possibly with GPUs.

A less resource-intensive approach is to entirely exhaust the chunk $\mathbf{A}$ before predicting a new chunk of actions, a strategy we refer to as *synchronous* (sync) inference. Moreover, sync inference efficiently allocates computation every n timestep, resulting in a reduced average computational burden at control time. In contrast, it inherently hinders the responsiveness of robot systems, introducing blind lags due to the robot being *idle* while computing $\mathbf{A}$.

We directly assess the lack of adaptiveness of robot systems due to acting open-loop, and the presence of lags at runtime by decoupling action chunk prediction $\mathbf{A}$ from action execution $a_t \leftarrow \text{POPFRONT}(\mathbf{A}_t)$, developing an *asynchronous* (async) inference stack (Algorithm 1), whereby a ROBOTCLIENT sends an observation o_t to a POLICYSERVER, receiving an action chunk $\mathbf{A}_t$ once inference is complete (Figure 2). In this, we avoid execution lags by triggering chunk prediction while the control loop is still consuming a previously available queue, aggregating it with the newly incoming queue whenever available. In turn, async-inference tightens the loop between action prediction and action execution, by increasing the frequency at which observations are processed for chunk prediction. Crucially, decoupling action prediction from action execution also directly allows to allocate more computational resources on a remote policy server sending actions to the robot client over networks, something which may prove very effective in resource-constrained scenarios such as low-power robots.

Algorithm 1 Asynchronous inference control-loop

```

1: Input: horizon  $T$ , chunk size  $n$ , threshold  $g \in [0, 1]$ 
2: Init: capture  $o_0$ ; send  $o_0$  to POLICYSERVER; receive  $\mathbf{A}_0 \leftarrow \pi(o_0)$ 
3: for  $t$  to  $T$  do
4:    $a_t \leftarrow \text{POPFRONT}(\mathbf{A}_t)$ 
5:   EXECUTE( $a_t$ ) ▷ execute action at step  $t$ 
6:   if  $\frac{|\mathbf{A}_t|}{n} < g$  then ▷ queue below threshold
7:     capture new observation,  $o_{t+1}$ 
8:     if NEEDSPROCESSING( $o_{t+1}$ ) then ▷ similarity filter, or triggers direct processing
9:        $\text{async\_handle} \leftarrow \text{ASYNCINFER}(o_{t+1})$  ▷ Trigger new chunk prediction (non blocking)
10:       $\hat{\mathbf{A}}_{t+1} \leftarrow \pi(o_{t+1})$  ▷ New queue is predicted with the policy
11:       $\mathbf{A}_{t+1} \leftarrow f(\mathbf{A}_t, \hat{\mathbf{A}}_{t+1})$  ▷ aggregate overlaps (if any)
12:    end if
13:  end if
14:  if NOTCOMPLETED( $\text{async\_handle}$ ) then ▷ No update on queue (inference is not over just yet)
15:     $\mathbf{A}_{t+1} \leftarrow \mathbf{A}_t$ 
16:  end if
17: end for

```

Action queue size ($|\mathcal{Q}|$) versus timesteps. Queue is consumed by RobotClient and refilled by PolicyServer

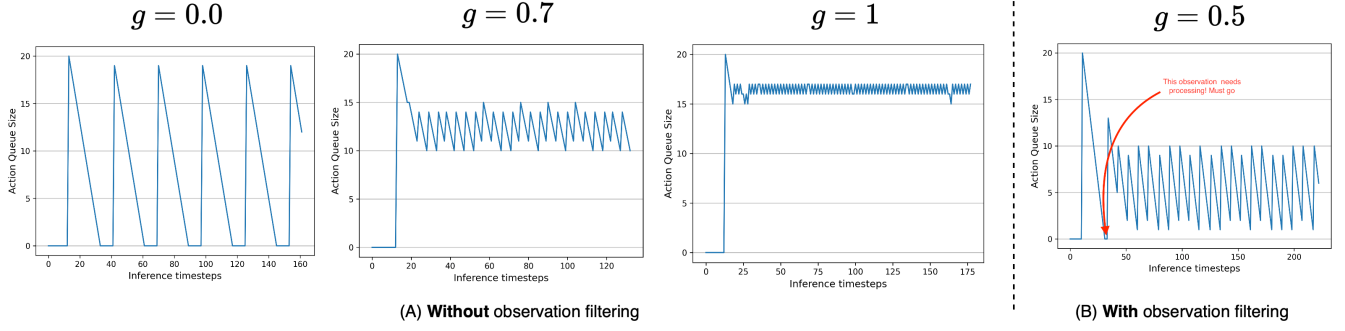


Figure 3 | Action queue size evolution at runtime for various levels of g when (A) not filtering out observation based on joint-space similarity and (B) filtering out near-duplicates observation, measuring their similarity in joint-space.

Implementation details *Async* inference (i) tightens the control loop by capturing observations more often, directly eliminates idle gaps at runtime, and (ii) directly allows to run inference on more powerful computational resources than the ones typically available onboard autonomous robotic platforms.

Algorithmically, we attain (i) on the ROBOTCLIENT-side by consuming actions from a readily available queue until a threshold condition on the number of remaining actions in the queue ($|\mathbf{A}_t|/n < g$) is met. When this condition is triggered, a new observation of the environment is captured and sent to the (possibly remote) POLICYSERVER. To avoid redundant server calls and erratic behavior at runtime observations are compared in joint-space, and near-duplicates are dropped. Two observations are considered near-duplicates if their distance in joint-space is under a predetermined threshold, $\epsilon \in \mathbb{R}_+$. Importantly, when the queue available to robot client eventually becomes empty, the most recent observation is processed regardless of similarity.

Interestingly, the behavior of async inference can be studied analytically. First, let ℓ be a random variable modeling the time needed to receive an action chunk $\mathbf{A}$ after sending an observation o , i.e. the sum of (i) the time to send across the observation o between the ROBOTCLIENT and POLICYSERVER, $t_{C \rightarrow S}$ (ii) the inference latency on the POLICYSERVER, ℓ_S and (iii) the time to send $\mathbf{A}$ between the POLICYSERVER and ROBOTCLIENT, $t_{S \rightarrow C}$. Assuming independence, $\mathbb{E}[\ell] = \mathbb{E}[t_{C \rightarrow S}] + \mathbb{E}[\ell_S] + \mathbb{E}[t_{S \rightarrow C}]$ which can be further simplified to $\mathbb{E}[\ell] \simeq \mathbb{E}[\ell_S]$, assuming communication time is (i) equal in both directions and (ii) negligible with respect to the inference latency. Second, let Δt be the environment’s control cycle. With a real-world frame-rate of 30 frames per second, $\Delta t = 33\text{ms}$. Consequently, exhausted queues at runtime—i.e. being idle awaiting for a new chunk—are avoided for $g \geq \frac{\mathbb{E}[\ell_S]/\Delta t}{n}$. In this, the queue threshold g plays a major role relatively to the availability of actions to the ROBOTCLIENT.

Figure 3(A) illustrates how the size of the action chunk $|\mathbf{A}_t|$ evolves over time for three representative values of g , detailing the following key scenarios:

- **Sequential limit** ($g = 0$). The client drains the entire chunk before forwarding a new observation to the server. During the round-trip latency needed to compute the next chunk, the queue is empty, leaving the robot *incapable of acting*. This reproduces the behavior of a fully sequential deployment and results in an average of $\mathbb{E}[\ell_S]$ idle seconds.
- **Asynchronous inference** ($g = 0.7$). Allowing the client to consume a fraction of roughly $1 - g = 0.3$ of a queue $\mathbf{A}_{t-1}$ before triggering inference for a new action queue $\mathbf{A}_t$, amortizing computation while keeping the queue from emptying. The overlap between successive chunks provides a buffer against modeling errors without the full cost of the $g = 1$ regime. The updated queue $\mathbf{A}_t$ is obtained aggregating queues on the overlapping timesteps between $\mathbf{A}_{t-1}$ and the incoming $\mathbf{A}_t$.
- **Compute-intensive limit** ($g = 1$). As an extreme case, and in keeping with Zhao et al. (2023); Chi et al. (2024), an observation is sent at *every* timestep. The queue is therefore almost always filled, with only a minor saw-tooth due to $\Delta t/\mathbb{E}[\ell_S] < 1$. While maximally reactive, this setting incurs one forward pass per control tick and can prove prohibitively expensive on limited hardware. Importantly, because the client is consuming actions while the server computes the next chunk, the available queue never gets filled again.

Figure 3(A) emphasizes the trade-off governed by g : small values place result in idle periods, whereas $g \approx 1$ assumes a highly accurate model and pays a significant compute price. In practice, choosing $g \in (0, 1)$ allows to strike a balance

between reactivity against resource budgets. If not for the aforementioned similarity filter, the `ROBOTCLIENT` would send observations for processing every $(1-g)n \cdot \Delta t$ seconds, receiving a new chunk of actions every $(1-g)n \cdot \Delta t + \mathbb{E}[\ell_S]$, on average. The presence of the observation similarity filter dilates this processing time, and serves the scope of avoiding the robot stalling due to the queue being constantly integrated with an incoming, nearly identical, action chunk. In particular, Figure 3(B) results in a queue which is filled with incoming actions *unless* near-duplicate observations are filtered out from the processing pipeline. For clarity, the red arrow in Figure 3(B) highlights a timestep where the observation similarity mechanism is bypassed, forcing a (nearly identical) observation to be processed as the queue results empty.

4 Experiments

4.1 Experimental setup

We evaluate our model on both simulated and real-world robotic manipulation tasks. To evaluate SmolVLA in simulation, we collected a new dataset for MetaWorld (Yu et al., 2020) comprising of 50 demonstrations for each of the 50 tasks. For real-world evaluation, we collected three datasets using the SO-100 robot arm (Knight et al.) and 1 with SO-101 arm (Knight et al.), each corresponding to a different manipulation task. Each dataset contains demonstrations relative to one task, with 10 trajectories for each of 5 distinct starting positions, resulting in a total of 50 demonstrations per dataset. The datasets record trajectories relative to Unless specified otherwise, SmolVLA is always trained in a multi-task setting.

Evaluation metrics. We report success rate (SR) as the primary metric across all benchmarks. For simulation-based evaluations, SR is binary—set to 1 if the task is successfully completed, and 0 otherwise. For real-world evaluations, we adopt a more fine-grained scoring approach by decomposing each task into subtasks. For example, in the Pick-and-Place task, we assign a score of 0.5 for successfully picking the cube and an additional 0.5 for correctly placing it into the target container.

Simulated environments. We evaluate SmolVLA in two established multi-task simulation benchmarks: LIBERO (Liu et al., 2023a) and Meta-World (Yu et al., 2020). LIBERO assesses diverse visuomotor skills across four categories—*Spatial*, *Object*, *Goal*, and *Long*—with 10 tasks per category (40 total). We use a dataset (Kim et al., 2024; Pertsch et al., 2025)¹ containing 1,693 episodes covering all tasks, and evaluate with 10 trials per task, reporting average success rates based on binary completion criteria. Meta-World evaluates generalization across 50 tasks of varying difficulty: *easy*, *medium*, *hard*, and *very hard* (Seo et al., 2023). We use a dataset² of 2,500 episodes (50 per task), and mirror the evaluation protocol used for LIBERO: 10 trials per task, with trials scored as 1 only if the task is fully completed.

Real-world tasks. We evaluated SmolVLA on 4 datasets in a real-world setting, which we open-source on Hugging Face (Figure 4). In particular, we benchmark real-world pick and placing capabilities³, stacking capabilities⁴, and sorting capabilities⁵ for the SO100 robot, alongside real-world pick and placing capabilities for the SO101 platform⁶. Critically, SmolVLA is not pretrained on any datasets recorded for the SO101.

For the pick and place task, SmolVLA is instructed to **pick up the cube and place it in the box**. The box is small in size and in a fixed position while the cube starting position is varied within 5 different starting conditions. We assess completion of the task with a fine-grained score resulting in a score of 0.5 for successfully grasping the cube, and 0.5 for successfully placing it into the box.

For the stacking task, SmolVLA is required to put a cube on top of another. We instruct the robot to **pick up the red cube and put it on top of the blue cube**. The initial positions of both cubes vary across episodes. We assess completion of the task with a fine-grained score resulting in a score of 0.5 for successfully grasping the top cube, and 0.5 for successfully placing it on top of the bottom cube.

¹LIBERO dataset: [physical-intelligence/libero](#)

²Meta-World dataset: [lerobot/metaworld_mt50](#)

³Pick-Place dataset: [lerobot/svla_so100_pickplace](#)

⁴Stacking dataset: [lerobot/svla_so100_stacking](#).

⁵Sorting dataset: [lerobot/svla_so100_sorting](#).

⁶(SO101) Pick-Place dataset: [lerobot/svla_so101_pickplace](#)

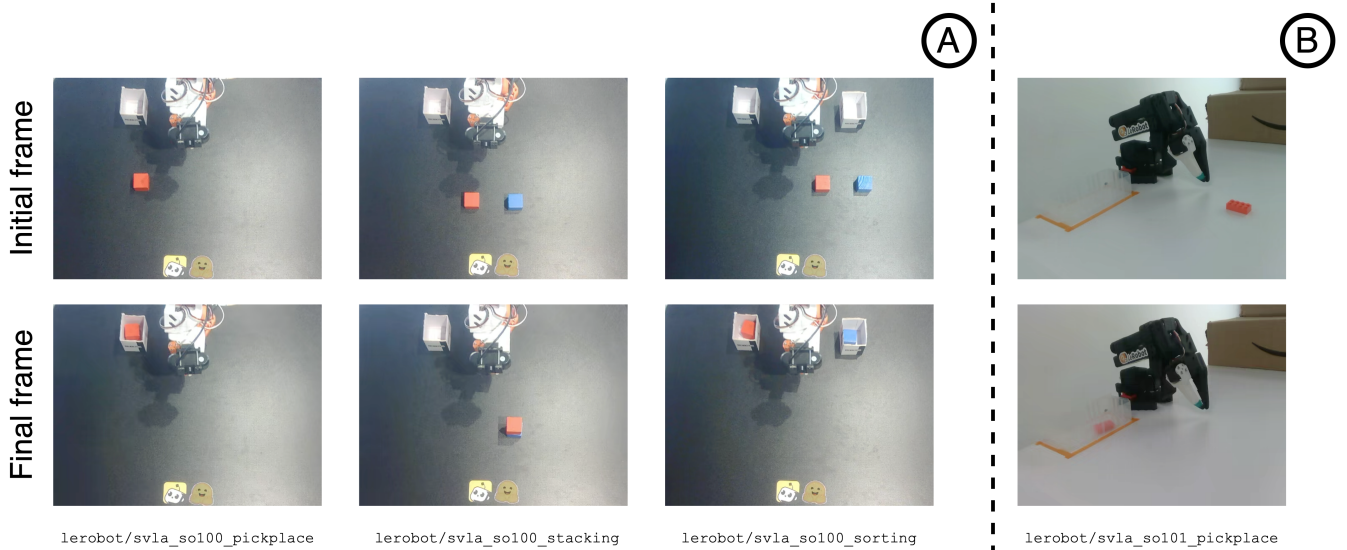


Figure 4 | Illustrations of the four real-world tasks we benchmark SmolVLA against, presenting starting and terminal frame for each of the dataset considered, for both SO100 embodiments (A) and SO101 (B). For SO100, we use top and wrist cameras, where for SO101 we use top and side cameras (as seen in the images).

For the sorting tasks, which has longer horizon, SmolVLA must sort the cubes depending on the color, following the instruction to put the red cube in the right box and the blue cube in the left box. The cubes are placed in 5 different positions as in Task 1. To introduce variation, the colors of the cubes are flipped, with 5 episodes per color configuration, resulting in 10 demonstrations per position. The boxes locations remain fixed across all demonstrations. We assess completion of the task with a fine-grained score resulting in a score of 0.25 for successfully grasping either of the cubes, and 0.25 for successfully completing one cube-box matching, resulting in a score of 0.25×4 upon task completion. Figure 4(A) presents initial and final frames for successful episodes for all tasks, alongside the Hugging Face handle of the corresponding dataset⁷.

To assess SmolVLA’s generalization, we also evaluate our model on a different robot embodiment and task⁸, similar to pick-place but rather using a small block instead of a cube. In this task, the robot is instructed to put the pink lego brick into the transparent box. This task requires more precision, especially in grasping the small lego object, together with advanced vision capabilities considering the box’s transparency.

4.2 Robots

Across simulation and real-world environments, we use a variety of robotic platforms.

- **SO100 and SO101 (Cadene et al., 2024)**. The Standard Open SO-100 is a low-cost, 3D-printable robotic arm designed for improving accessibility to robotics and robot learning research. Both the SO-100 and its updated version, the SO-101, are open-source platforms for basic manipulation tasks. Each arm has six degrees of freedom and uses low-cost servo motors that are controlled with position commands. The SO101 has better arm design for faster assembly and different motors, making its movements smoother and better for tasks requiring more precisions.
- **Panda (Haddadin et al., 2022)**. The Franka Emika Panda is a single 7-DOF torque-controlled robotic arm designed for safe and precise manipulation. Its high-resolution joint sensing and compliant control make it well-suited for learning-based manipulation tasks in both simulation and real-world settings. This robot is used in the LIBERO simulator.
- **Swayer (Yu et al., 2020)**. Is a single 4-DOF controlled robotic arm designed for manipulation tasks. Is is used in the Meta-World simulator and the policy control the position and state of the gripper.

⁷Datasets can be easily explored via `visualize_dataset`

⁸Pick-Place-Lego dataset: `lerobot/svla_so101_pickplace`.

4.3 Implementation details.

We conduct our experiments using LeRobot (Cadene et al., 2024), a PyTorch-based framework for real-world robotics. During pretraining, we train for 200,000 steps with a global batch size of 256 on all our community datasets. After a 100-step warmup, we use a cosine learning rate schedule starting at $1e-4$ and decaying to a minimum of $2.5e-6$. We use the AdamW optimizer with $\beta_1 = 0.9, \beta_2 = 0.95$. Training is performed after resizing the images to 512×512 , for consistency with the VLM input size. We use SmolVLM-2 (Marafioti et al., 2025) as our VLM backbone. The action expert is trained with flow matching to output chunks of $n = 50$ actions. For real-world evaluation, we perform synchronous inference: the model samples new observations only after executing the full chunk of actions. In simulation, we perform inference by sampling new observations and predicting a new action after each executed action. During inference, the flow matching is fixed to 10 steps. We train only the action expert module, keeping the VLM frozen. Our main model, contains 450 million parameters, with approximately 100 million dedicated to the action expert. We use only the first 16 layers of the large language model (LLM) within the VLM. For fine-tuning on simulation benchmarks, we train for 100,000 steps with a batch size of 64, while for real-world tasks, we fine-tune for 200,000 steps. However, we observe in practice that the model can be trained for a much smaller number of steps without sacrificing significant performance levels.

Beyond maintaining a compact model and a reduced number of tokens, we employ several optimizations to enhance training efficiency. Specifically, we leverage `bfloat16` precision and `torch.compile()` (Paszke, 2019) that JIT-compiles PyTorch code into optimized kernels. To ensure compatibility with these optimizations, we maintain a fixed sequence length and batch size, discarding any excess frames in an episode that do not fit a complete batch. For multi-GPU and multi-node training, we utilize Hugging Face’s `accelerate` (Gugger et al., 2022) library with mixed precision, providing a scalable and memory-efficient training setup. Pretraining was conducted using 4 GPUs to accommodate for large batch size, but the model can easily be trained on a single GPU due to its small size. Overall, the project consumed approximately 30k GPU hours.

4.4 Baselines

We compare our model against two popular and strong baselines, both available in the LeRobot library (Cadene et al., 2024).

π_0 (Black et al., 2024). π_0 is a VLA which leverages a VLM combined with Flow Matching for action chunk prediction. It has a total model size of 3.3 billion parameters and is pre-trained on 10,000 hours of cross-embodiment robotics data. The model architecture is based on Paligemma (Beyer et al., 2024) and accepts three RGB images, sensorimotor states, and a language instruction as inputs.

ACT (Zhao et al., 2023). ACT is a Conditional Variational Autoencoder (CVAE) (Sohn et al., 2015) policy model with an encoder-decoder transformer architecture containing approximately 80 million parameters. ACT uses a ResNet vision encoder pre-trained on ImageNet, while the CVAE is trained from scratch. The model generates action chunks and is optimized using a regression objective, directly predicting continuous actions. The model accepts a sequence of RGB images, and sensorimotor states.

4.5 Main results

In this section, we present the main results of SmolVLA in both real-world and simulated environments. For real-world evaluation, SmolVLA is pretrained on community-collected datasets. π_0 is finetuned on the respective target datasets, while ACT is trained from scratch on each dataset.

Simulation Evaluation. In Table 2, we further evaluate SmolVLA on two major simulation benchmarks—LIBERO and Meta-World—using a multi-task training setup. SmolVLA outperforms other VLA-based approaches such as Octo (Team et al., 2024) and OpenVLA (Kim et al., 2024), as well as the diffusion policy baseline across both LIBERO and Meta-World. We also compare against two variants of π_0 : one initialized from a vision-language model (Paligemma-3B), and another further pretrained on robotics datasets (initialized from the weights released by the authors). Despite not being pretrained on robotics data, SmolVLA consistently outperforms the VLM-initialized π_0 and performs competitively with the robotics-pretrained version. Note that, compared to π_0 , SmolVLA is around 40% faster to train and consumes 6× less memory.

Benchmark	Policy (# Params)	VLA Pt.	Success Rate (%) – Simulation				
LIBERO			Spatial	Object	Goal	Long	Avg.
	Diffusion Policy (Khazatsky et al., 2024)	No	78.3	92.5	68.3	50.5	72.4
	Octo (0.09B) (Team et al., 2024)	Yes	78.9	85.7	84.6	51.1	75.1
	OpenVLA (7B) (Kim et al., 2024)	Yes	84.7	88.4	79.2	53.7	76.5
	π_0 (Paligemma-3B)	No	87	63	89	48	71.8
	π_0 (3.3B)	Yes	90	86	95	73	86.0
	SmolVLA (0.24B)	No	87	93	88	63	82.75
	SmolVLA (0.45B)	No	90	96	92	71	87.3
	SmolVLA (2.25B)	No	93	94	91	77	88.75
Meta-World			Easy	Medium	Hard	Very Hard	Avg.
	Diffusion Policy (Chi et al., 2023)	No	23.1	10.7	1.9	6.1	10.5
	TinyVLA (Zhou et al., 2024)	No	77.6	21.5	11.4	15.8	31.6
	π_0 (3.5B-Paligemma)	No	80.4	40.9	36.7	44.0	50.5
	π_0 (3.5B)	Yes	71.8	48.2	41.7	30.0	47.9
	SmolVLA (0.24B)	No	86.43	46.36	35	60	56.95
	SmolVLA (0.45B)	No	82.5	41.8	45.0	60.0	57.3
	SmolVLA (2.25B)	No	87.14	51.82	70	64	68.24

Table 2 | Simulation benchmarks (LIBERO and Meta-World). Success rates (%) for various policies. LIBERO tasks correspond to different settings (e.g., spatial, object, goal, long-horizon); Meta-World tasks reflect different difficulty levels. VLA Pt refers pretraining on robotics data. SmolVLA is only initialized from the VLM. Baselines scores from Kim et al. (2024); Wen et al. (2024).

Policy	Success Rate (%) – Real World			
	Pick-Place	Stacking	Sorting	Avg.
Single-task Training				
ACT	70	50	25	48.3
Multi-task Training				
π_0 (3.5B)	100	40	45	61.7
SmolVLA (0.45B)	75	90	70	78.3

Table 3 | Real-world benchmarks (SO100). Success rate (%) across three tasks using policies trained in multi-task and single-task settings.

Policy	Success Rate (%) – Real World	
	In Distribution	Out of Distribution
Single-task Training		
ACT	70	40
SmolVLA (0.45B)	90	50

Table 4 | Real-world benchmark (SO101). Success rate (%) for the Pick-Place-Lego task using policies trained in single-task setting.

Real-World Evaluation. In Table 3, we evaluate SmolVLA on four real-world tasks. For the SO101 benchmark, the model is trained on a combination of three datasets, and success rates are reported per task as well as on average. SmolVLA outperforms both ACT (Zhao et al., 2023), which is trained individually on each task, and π_0 , a significantly larger model in terms of parameter count ($\sim 7\times$). Similarly, on SO101 (see Table 4), SmolVLA surpasses ACT in both in-distribution and out-of-distribution (OOD) settings. For OOD evaluation, the Lego object is placed in novel positions not previously encountered during training.

Effect of pretraining and multitask learning. In Table 5, we further evaluate the impact of pretraining SmolVLA on community datasets in terms of the difference in real-world performance, and investigate whether multitask finetuning provides additional benefits for SmolVLA. The results show that, pretraining on community datasets leads to a substantial performance improvement (from 51.7 to 78.3). Furthermore, multitask finetuning yields further gains, underscoring the importance of knowledge transfer across tasks.

Policy	VLA pt.	Success Rate (%) – Real World			
		Pick-Place	Stacking	Sorting	Avg.
Single-task Training					
SmolVLA (0.45B)	No	55	45	20	40
Multi-task Training					
SmolVLA (0.45B)	No	80	40	35	51.7
SmolVLA (0.45B)	Yes	75	90	70	78.3

Table 5 | Effect of pretraining and multitask learning. Success rate (%) across three tasks using SmolVLA trained in multi-task and single-task settings. Results with SO100 robot.

Inference	Success Rate (%) – Real World				Inference	Time (s) – Real World			Inference	# of Cubes – Real World		
	Pick-Place	Stacking	Sorting	Avg		Total	Avg	Std		Total	Avg	Std
Sync	75	90	70	78.3	Sync	137.5	13.75	2.42	Sync	9	1.8	0.45
Async	80	90	50	73.3	Async	97.0	9.70	2.95	Async	19	3.8	1.3

(a) | Performance (success rates). (b) | Task completion time. (c) | Performance in fixed time.

Figure 5 | Comparison between synchronous (Sync) and asynchronous (Async) inference. We evaluate SmolVLA on three real-world tasks under both inference modes. Asynchronous inference achieves similar success rates (left) but is significantly faster (middle) and complete more tasks (right) in fixed-time settings. Hyperparameters have been optimized for Pick-Place and reused across tasks.

4.6 Asynchronous inference

We evaluated SmolVLA under two inference modes: sync and async. The sync mode reflects a standard evaluation setting in robotics, whereby the policy predicts a chunk of actions that is executed fully before the next prediction cycle begins. In contrast, the async mode decouples action execution and policy inference, allowing predictions and control to run in parallel.

Results. For both inference modes, we report the success rate and policy speed (Figure 5). To evaluate speed, we design two experiments using the Pick-Place task. In the first experiment, we measure the time taken to complete the task across 10 trials and 5 different cube positions. In the second, we fix a time limit (e.g., 60 seconds) and count the number of cubes successfully picked and placed into the box, from different positions. Timing begins when the robot starts moving. As shown in Figure 5a, both inference modes achieve comparable success rates across three real-world tasks. However, asynchronous inference demonstrates a substantial speed advantage (Figure 5b). On average, it completes the task in 9.7 seconds, compared to 13.75 seconds in the synchronous setting (~ 30% faster). Furthermore, under the fixed-time evaluation, the asynchronous mode allows the robot to complete 19 successful pick-and-place cycles, compared to only 9 in the synchronous case (Figure 5c). Qualitatively, we observe that asynchronous inference enables faster reactions and better adaptability to changes in the environment. The robot exhibits greater robustness to shifts in object positions and external disturbances, and overall is capable to solve the same tasks a significantly larger number of times due to the avoidance of prediction lags (Figure 5).

4.7 Ablation Study

We conduct a comprehensive ablation study to assess key design choices behind the final SmolVLA model. All ablations are conducted on the LIBERO benchmark. Unless otherwise noted, models are trained from scratch without any pretraining on robotics data. The VLM backbone is frozen, and only the action expert is trained from scratch.

Cross-attention (CA) vs. self-attention (SA) between VLM and $\mathbf{v}_\theta$. We compare how the VLM features interact with the action expert, comparing causal self-attention (SA), cross-attention (CA), or our proposed interleaved SA+CA setup. In the SA setting, action tokens attend to each other using a causal mask, while in the CA setting the VLM features act as keys and values for attention in the $\mathbf{v}_\theta$. As shown in Table 6, cross-attention outperforms self-attention significantly. Interleaving both yields the best results, highlighting their complementary strengths.

Attention mechanism	Success Rate (%) – LIBERO				
	S	O	G	10	Avg
CA	87	92	83	54	79.0
SA	80	94	84	40	74.5
CA+SA (ours)	86	99	90	67	85.5

Table 6 | Cross vs self-attention. Interleaved cross (CA) and self-attention (SA) yield the best results.

N	Success Rate (%) – LIBERO				
	S	O	G	10	Avg
8	77	88	86	49	75.0
16	88	91	91	44	78.5
24	86	97	86	49	79.5
32	89	94	85	53	80.3
Skip %2	84	90	83	45	75.5
VLM-256M	86	83	75	59	75.8

Table 8 | Skipping VLM layers. Skipping layers from a large VLM (here 500M parameters) yields better results than downsizing the VLM. Skipping every second layer is a competitive baseline.

Attention mask	Success Rate (%) – LIBERO				
	S	O	G	10	Avg
Bidir	79	86	82	23	67.5
Causal	80	94	84	40	74.5

Table 7 | Bidirectional vs causal attention. Preventing future action leakage improves performance.

Expert width (w.r.t. VLM)	Success Rate (%) – LIBERO				
	S	O	G	10	Avg
$\times 1.00$	87	96	90	56	82.3
$\times 0.75$	82	89	84	55	77.5
$\times 0.50$	89	94	85	53	80.3
$\times 0.25$	76	97	83	39	73.8

Table 9 | Expert capacity. Adjusting the expert’s hidden size affects performance. Larger capacities yield better success rates.

Causal vs. bidirectional attention on action tokens within $\mathbf{v}_\theta$. Next, we investigate how action tokens should attend to each other within the action expert, $\mathbf{v}_\theta$. We compare: (i) no interaction between action tokens (pure CA), (ii) causal self-attention, and (iii) bidirectional self-attention. Table 7 shows causal self-attention performs best, while bidirectional interaction harms performance. Surprisingly, the no-interaction (CA-only) setup performs competitively, suggesting that conditioning on VLM features alone can be powerful.

Using early LLM layers in the VLM. The VLM backbone consists of a vision encoder followed by an LLM. Motivated by improving the efficiency of SmolVLA, we investigate using features from the first $N < L$ layers only instead of all the L LLM layers available or the features (Black et al., 2024). Before starting to train, we discard the top $L - N$ layers of the VLM. As shown in Table 8, using only the first half of the VLM layers gives a good trade-off between performance and compute. Further, we also test a variant sampling every second VLM layer (Skip % 2, (Shukor and Cord, 2024))—reducing depth by half while retaining full model capacity. Table 8 indicates skipping every second layer performs better than training a smaller VLM, but worse than using the first $N < L$ layers directly.

Action Expert Capacity. Motivated by efficiency arguments, we investigate varying the hidden dimension of the action expert to explore the impact of model capacity on performance. Given the VLM dimension d , Table 9 shows reducing the expert’s hidden size to $0.75 \times d$ achieves a good balance between performance and efficiency.

Regression vs. Flow Matching training objectives. We compare two learning objectives for training the action expert $\mathbf{v}_\theta$: flow matching (our default), and a standard regression L1 loss on the predicted versus ground-truth action chunks. In keeping with Black et al. (2024); Chi et al. (2024) Table 10 shows that flow matching significantly outperforms regression, suggesting flow matching provides better inductive bias for modeling complex, multimodal action distributions.

States to the VLM or Action Expert? We compare two variants: (i) feeding the sensorimotor states to the VLM (by projecting them into token space), and (ii) passing them directly to the action expert. Table 11 indicates including state information in the VLM leads to significantly better performance for both the CA and SA variants.

Training objective	Success Rate (%) – LIBERO				
	S	O	G	10	Avg
Flow matching	89	94	85	53	80.25
Regression	92	85	86	38	75.25

Table 10 | Training objective. We compare our training objective based on Flow matching to regression with L1 loss.

Chunk Size	Success Rate (%) – LIBERO				
	S	O	G	10	Avg
1	45	77	54	24	50.0
10	90	94	94	58	84.0
30	85	94	87	48	78.5
50	89	94	85	53	80.3
100	83	88	85	42	74.5

Table 12 | Action chunk size. A chunk size between 10 and 50 strikes a good balance between action prediction frequency and performance.

Action chunk size, n . Our model predicts chunks of actions, where each chunk consists of n time steps. We study the effect of varying n on the overall performance. A larger n allows the robot to execute more actions at inference time before needing to process new observations and predict the next chunk. However, Table 12 shows that both very small and very large values of n degrade performance. We find that chunk sizes between 10 and 50 provide a good balance between the robot reactivity and efficiency.

Number of executed actions before updating observations. To improve inference speed in real-world deployment, the robot can execute multiple actions from the predicted chunk before processing new observations, hereby overwriting the current chunk before its exhaustion. Still, while acting the entire chunk speeds up inference, it also reduces the robot’s responsiveness to environmental changes. Table 13 demonstrates updating observations more frequently significantly improves success rate, highlighting a trade-off between inference speed and control accuracy.

5 Discussion

We introduce a compact, efficient, and lightweight VLA model–SmolVLA–that runs on consumer-grade hardware, controls low-cost robots, and rivals significantly larger VLAs. SmolVLA’s architecture is designed for efficient training and inference without compromising the success rate. In addition, we propose an asynchronous inference stack that enables faster adaptation and responsiveness in real-world manipulation tasks. This inference strategy is model-agnostic and can be integrated with any policy that outputs chunks of actions. Our work is supported by thorough ablations and analysis of the proposed architecture, that can guide practitioners and researchers to further improve the model architecture. Finally, we open-source our model, codebase, training datasets, robots hardware and provide detailed instructions to facilitate full reproducibility.

5.1 Limitations

We identify several limitations remaining in our contribution. In particular:

- **Dataset diversity and cross-embodiment training.** Our pretraining currently uses datasets collected from a single robot type (SO100). Although we demonstrate that the model can be fine-tuned to different robots (Table 4) and

States	Attention	Success Rate (%) – LIBERO				
		S	O	G	10	Avg
Prefix	CA	89	94	85	53	80.3
Suffix	CA	86	82	78	47	73.3
Prefix	SA	62	74	57	20	53.3
Suffix	SA	80	92	80	47	74.8

Table 11 | States as prefix vs. suffix. Feeding states to the VLM (prefix) leads to better performance than feeding them to the expert (suffix).

Action Steps	Success Rate (%) – LIBERO				
	S	O	G	10	Avg
1	89	94	85	53	80.3
10	89	94	91	57	82.8
30	76	91	74	42	70.8
50	54	70	58	25	51.8

Table 13 | Action execution steps. Sampling new observations more frequently (e.g., every 1 or 10 steps) significantly improves performance.

outperforms existing baselines, we argue incorporating training data from multiple robot embodiments is likely to prove critical in enhancing the model’s ability to generalize to new robotic platforms.

- **Dataset size and scalability.** The dataset used for training contains approximately 23k trajectories, and is significantly smaller than those used in typical VLA training regimes—OpenVLA, for instance, utilizes around 1 million trajectories. Expanding the dataset size could substantially improve the model’s performance and its generalization across a wider range of tasks and environments.
- **Model size and hardware efficiency.** SmolVLA has less than 0.5 billion parameters, allowing for fast inference on consumer-grade hardware. While this efficiency is beneficial, exploring ways to scale these architectures further without sacrificing speed or accessibility is an important direction for future research.
- **Choice of VLM backbone.** We rely on an off-the-shelf VLM backbone pretrained mainly on document reading and OCR tasks (Marafioti et al., 2025). However, it is not yet clear whether these VLMs are optimal for real-world robotic interaction scenarios. Future work could explore alternative or more specialized pretraining strategies to better align VLM backbones with the peculiar demands of robotic environments.
- **Multimodal and robotics data joint training.** Integrating shared training on both robotics-specific data and broader multimodal datasets has the potential to improve generalization and instruction-following abilities. Such joint training could lead to more robust and adaptable VLAs.
- **Task complexity and longer horizon.** While SmolVLA competes effectively on relatively simple and short-horizon tasks, scaling the approach to tackle longer-horizon problems remains an important challenge. Incorporating hierarchical policies or multi-level planning mechanisms may help to address this complexity.
- **Learning paradigms: Imitation vs. Reinforcement Learning.** Our current approach primarily relies on imitation learning. Nevertheless, exploring reinforcement learning techniques for VLAs (Chen et al., 2025)—especially for handling complex or long-horizon tasks—could offer significant performance benefits and more dexterous policy adaptation.

6 Acknowledgements

The authors thank Marina Barannikov, Alexandre Chapin, Ville Kuosmanen, and Jade Choghari for their help in community and simulated datasets. The authors thank Alexander Soare for his early work on asynchronous inference. The authors thank Quentin Gallouedec, Pablo Montalvo-Leroux and the rest of the Hugging Face team for their support during this project. This work was partly supported by the HPC resources of IDRIS under the allocation 2025-[A0181016235] made by GENCI and the PostGenAI@Paris cluster (ANR-23-IACL-0007, FRANCE 2030).

References

- Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.
- AI@Meta. Llama 3.2 model card, 2024. https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md.
- Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in neural information processing systems*, 35:23716–23736, 2022.
- Loubna Ben Allal, Anton Lozhkov, Elie Bakouch, Gabriel Martín Blázquez, Guilherme Penedo, Lewis Tunstall, Andrés Marafioti, Hynek Kydlíček, Agustín Piqueres Lajarín, Vaibhav Srivastav, et al. Smollm2: When smol goes big—data-centric training of a small language model. *arXiv preprint arXiv:2502.02737*, 2025.
- Shuai Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, Sibao Song, Kai Dang, Peng Wang, Shijie Wang, Jun Tang, Humen Zhong, Yanzhi Zhu, Mingkun Yang, Zhaohai Li, Jianqiang Wan, Pengfei Wang, Wei Ding, Zheren Fu, Yiheng Xu, Jiabo Ye, Xi Zhang, Tianbao Xie, Zesen Cheng, Hang Zhang, Zhibo Yang, Haiyang Xu, and Junyang Lin. Qwen2.5-vl technical report, 2025. <https://arxiv.org/abs/2502.13923>.
- Rohan Bavishi, Erich Elsen, Curtis Hawthorne, Maxwell Nye, Augustus Odena, Arushi Somani, and Sağnak Taşlılar. Introducing our multimodal models, 2023. <https://www.adept.ai/blog/fuyu-8b>.
- Lucas Beyer, Andreas Steiner, André Susano Pinto, Alexander Kolesnikov, Xiao Wang, Daniel Salz, Maxim Neumann, Ibrahim Alabdulmohsin, Michael Tschannen, Emanuele Bugliarello, et al. Paligemma: A versatile 3b vlm for transfer. *arXiv preprint arXiv:2407.07726*, 2024.

- Johan Bjorck, Fernando Castañeda, Nikita Cherniadev, Xingye Da, Runyu Ding, Linxi Fan, Yu Fang, Dieter Fox, Fengyuan Hu, Spencer Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. $\pi 0$: A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2503.24164*, 2024.
- Daniel Bolya, Po-Yao Huang, Peize Sun, Jang Hyun Cho, Andrea Madotto, Chen Wei, Tengyu Ma, Jiale Zhi, Jathushan Rajasegaran, Hanoona Rasheed, et al. Perception encoder: The best visual embeddings are not at the output of the network. *arXiv preprint arXiv:2504.13181*, 2025.
- Zalán Borsos, Raphaël Marinier, Damien Vincent, Eugene Kharitonov, Olivier Pietquin, Matt Sharifi, Dominik Roblek, Olivier Teboul, David Grangier, Marco Tagliasacchi, et al. Audioldm: a language modeling approach to audio generation. *IEEE/ACM transactions on audio, speech, and language processing*, 31:2523–2533, 2023.
- Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. Language models are few-shot learners. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin, editors, *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020. https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfc4967418bfb8ac142f64a-Paper.pdf.
- Minwoo Byeon, Beomhee Park, Haecheon Kim, Sungjun Lee, Woonhyuk Baek, and Saehoon Kim. Coyo-700m: Image-text pair dataset. <https://github.com/kakaobrain/coyo-dataset>, 2022.
- Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascale, Jade Choghari, Jess Moss, and Thomas Wolf. Lerobot: State-of-the-art machine learning for real-world robotics in pytorch. <https://github.com/huggingface/lerobot>, 2024.
- Xi Chen, Xiao Wang, Soravit Changpinyo, AJ Piergiovanni, Piotr Padlewski, Daniel Salz, Sebastian Goodman, Adam Grycner, Basil Mustafa, Lucas Beyer, et al. Pali: A jointly-scaled multilingual language-image model. *arXiv preprint arXiv:2209.06794*, 2022.
- Xi Chen, Xiao Wang, Lucas Beyer, Alexander Kolesnikov, Jialin Wu, Paul Voigtlaender, Basil Mustafa, Sebastian Goodman, Ibrahim Alabdulmohsin, Piotr Padlewski, et al. Pali-3 vision language models: Smaller, faster, stronger. *arXiv preprint arXiv:2310.09199*, 2023.
- Yuhui Chen, Shuai Tian, Shugao Liu, Yingting Zhou, Haoran Li, and Dongbin Zhao. Conrft: A reinforced fine-tuning method for vlm models via consistency policy. *arXiv preprint arXiv:2502.05450*, 2025.
- Zhe Chen, Weiyun Wang, Yue Cao, Yangzhou Liu, Zhangwei Gao, Erfei Cui, Jinguo Zhu, Shenglong Ye, Hao Tian, Zhaoyang Liu, et al. Expanding performance boundaries of open-source multimodal models with model, data, and test-time scaling. *arXiv preprint arXiv:2412.05271*, 2024.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, page 02783649241273668, 2023.
- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *The International Journal of Robotics Research*, 2024.
- Wenliang Dai, Junnan Li, Dongxu Li, Anthony Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. InstructBLIP: Towards general-purpose vision-language models with instruction tuning. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. <https://openreview.net/forum?id=vvoWPYqZJA>.
- Nilaksh Das, Saket Dingliwal, Srikanth Ronanki, Rohit Paturi, Zhaocheng Huang, Prashant Mathur, Jie Yuan, Dhanush Bekal, Xing Niu, Sai Muralidhar Jayanthi, et al. Speechverse: A large-scale generalizable audio language model. *arXiv preprint arXiv:2405.08295*, 2024.
- Alexandre Défossez, Laurent Mazaré, Manu Orsini, Amélie Royer, Patrick Pérez, Hervé Jégou, Edouard Grave, and Neil Zeghidour. Moshi: a speech-text foundation model for real-time dialogue. *arXiv preprint arXiv:2410.00037*, 2024.

- Haiwen Diao, Yufeng Cui, Xiaotong Li, Yueze Wang, Huchuan Lu, and Xinlong Wang. Unveiling encoder-free vision-language models. *arXiv preprint arXiv:2406.11832*, 2024.
- Haiwen Diao, Xiaotong Li, Yufeng Cui, Yueze Wang, Haoge Deng, Ting Pan, Wenxuan Wang, Huchuan Lu, and Xinlong Wang. Evv2: Improved baselines for encoder-free vision-language models. *arXiv preprint arXiv:2502.06788*, 2025.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- Frederik Ebert, Yanlai Yang, Karl Schmeckpeper, Bernadette Bucher, Georgios Georgakis, Kostas Daniilidis, Chelsea Finn, and Sergey Levine. Bridge data: Boosting generalization of robotic skills with cross-domain datasets. *arXiv preprint arXiv:2109.13396*, 2021.
- Alaaeldin El-Nouby, Michal Klein, Shuangfei Zhai, Miguel Angel Bautista, Alexander Toshev, Vaishaal Shankar, Joshua M Susskind, and Armand Joulin. Scalable pre-training of large autoregressive image models. *arXiv preprint arXiv:2401.08541*, 2024.
- Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first international conference on machine learning*, 2024.
- Enrico Fini, Mustafa Shukor, Xiujuan Li, Philipp Dufter, Michal Klein, David Haldimann, Sai Aitharaju, Victor Guilherme Turrissi da Costa, Louis Béthune, Zhe Gan, Alexander T Toshev, Marcin Eichner, Moin Nabi, Yinfei Yang, Joshua M. Susskind, and Alaaeldin El-Nouby. Multimodal autoregressive pre-training of large vision encoders, 2024.
- Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab Mangrulkar, Marc Sun, and Benjamin Bossan. Accelerate: Training and inference at scale made simple, efficient and adaptable. <https://github.com/huggingface/accelerate>, 2022.
- Sami Haddadin, Sven Parusel, Lars Johannismeier, Saskia Golz, et al. The franka emika robot: A reference platform for robotics research and education. *IEEE Robotics & Automation Magazine*, 29(2):2–20, 2022. doi: 10.1109/MRA.2021.3138382.
- Nicklas Hansen, Xiaolong Wang, and Hao Su. Temporal difference learning for model predictive control. In *ICML*, 2022.
- Jiangyong Huang, Silong Yong, Xiaojian Ma, Xiongkun Linghu, Puhao Li, Yan Wang, Qing Li, Song-Chun Zhu, Baoxiong Jia, and Siyuan Huang. An embodied generalist agent in 3d world. In *Proceedings of the 41st International Conference on Machine Learning*, pages 20413–20451, 2024.
- Shaohan Huang, Li Dong, Wenhui Wang, Yaru Hao, Saksham Singhal, Shuming Ma, Tengchao Lv, Lei Cui, Owais Khan Mohammed, Barun Patra, et al. Language is not all you need: Aligning perception with language models. *Advances in Neural Information Processing Systems*, 36:72096–72109, 2023.
- Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.
- Alexander Khazatsky, Karl Pertsch, Suraj Nair, Ashwin Balakrishna, Sudeep Dasari, Siddharth Karamcheti, Soroush Nasiriany, Mohan Kumar Srirama, Lawrence Yunliang Chen, Kirsty Ellis, et al. Droid: A large-scale in-the-wild robot manipulation dataset. *arXiv preprint arXiv:2403.12945*, 2024.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan P Foster, Pannag R Sanketi, Quan Vuong, et al. Openvla: An open-source vision-language-action model. In *8th Annual Conference on Robot Learning*, 2024.
- Rob Knight, Pepijn Kooijmans, Thomas Wolf, Simon Alibert, Michel Aractingi, Dana Aubakirova, Adil Zouitine, Russi Martino, Steven Palma, Caroline Pascal, and Remi Cadene. Standard Open SO-100 & SO-101 Arms. <https://github.com/TheRobotStudio/SO-ARM100>.
- Jing Yu Koh, Ruslan Salakhutdinov, and Daniel Fried. Grounding language models to images for multimodal inputs and outputs, 2023.
- Zhifeng Kong, Arushi Goel, Rohan Badlani, Wei Ping, Rafael Valle, and Bryan Catanzaro. Audio flamingo: A novel audio language model with few-shot learning and dialogue abilities. In *International Conference on Machine Learning*, pages 25125–25148. PMLR, 2024.
- Vik Korrapati. Moondream. Online, 2024. <https://moondream.ai/>. Accessed: 2025-03-27.
- Hugo Laurençon, Lucile Saulnier, Leo Tronchon, Stas Bekman, Amanpreet Singh, Anton Lozhkov, Thomas Wang, Siddharth Karamcheti, Alexander M Rush, Douwe Kiela, Matthieu Cord, and Victor Sanh. OBELICS: An open web-scale filtered dataset of interleaved image-text documents. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. <https://openreview.net/forum?id=SKN2hf1BIZ>.

- Hugo Laurençon, Léo Tronchon, Matthieu Cord, and Victor Sanh. What matters when building vision-language models? *arXiv preprint arXiv:2405.02246*, 2024.
- Seungjae Lee, Yibin Wang, Haritheja Etukuru, H Jin Kim, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. Behavior generation with latent actions. *arXiv preprint arXiv:2403.03181*, 2024.
- Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proceedings of the 40th International Conference on Machine Learning*, ICML’23. JMLR.org, 2023.
- Xinghang Li, Minghuan Liu, Hanbo Zhang, Cunjun Yu, Jie Xu, Hongtao Wu, Chilam Cheang, Ya Jing, Weinan Zhang, Huaping Liu, et al. Vision-language foundation models as effective robot imitators. In *ICLR*, 2024.
- Ji Lin, Hongxu Yin, Wei Ping, Yao Lu, Pavlo Molchanov, Andrew Tao, Huizi Mao, Jan Kautz, Mohammad Shoeybi, and Song Han. Vila: On pre-training for visual language models. *arXiv preprint arXiv:2312.07533*, 2023a.
- Xi Victoria Lin, Akshat Shrivastava, Liang Luo, Srinivasan Iyer, Mike Lewis, Gargi Ghosh, Luke Zettlemoyer, and Armen Aghajanyan. Moma: Efficient early-fusion pre-training with mixture of modality-aware experts. *arXiv preprint arXiv:2407.21770*, 2024.
- Ziyi Lin, Chris Liu, Renrui Zhang, Peng Gao, Longtian Qiu, Han Xiao, Han Qiu, Chen Lin, Wenqi Shao, Keqin Chen, et al. Sphinx: The joint mixing of weights, tasks, and visual embeddings for multi-modal large language models. *arXiv preprint arXiv:2311.07575*, 2023b.
- Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.
- Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023a.
- Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. In *NeurIPS 2023 Workshop on Instruction Tuning and Instruction Following*, 2023b. <https://openreview.net/forum?id=yx3Hkx5ved>.
- Jiajun Liu, Yibing Wang, Hanghang Ma, Xiaoping Wu, Xiaoqi Ma, Xiaoming Wei, Jianbin Jiao, Enhua Wu, and Jie Hu. Kangaroo: A powerful video-language model supporting long-context video input. *arXiv preprint arXiv:2408.15542*, 2024.
- Qiang Liu. Rectified flow: A marginal preserving approach to optimal transport. *arXiv preprint arXiv:2209.14577*, 2022.
- Oscar Mañas, Pau Rodriguez Lopez, Saba Ahmadi, Aida Nematzadeh, Yash Goyal, and Aishwarya Agrawal. MAPL: Parameter-efficient adaptation of unimodal pre-trained models for vision-language few-shot prompting. In Andreas Vlachos and Isabelle Augenstein, editors, *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2523–2548, Dubrovnik, Croatia, May 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.eacl-main.185. <https://aclanthology.org/2023.eacl-main.185>.
- Andrés Marafioti, Orr Zohar, Miquel Farré, Merve Noyan, Elie Bakouch, Pedro Cuenca, Cyril Zakka, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, et al. Smolvlm: Redefining small and efficient multimodal models. *arXiv preprint arXiv:2504.05299*, 2025.
- Abby O’Neill, Abdul Rehman, Abhiram Maddukuri, Abhishek Gupta, Abhishek Padalkar, Abraham Lee, Acorn Pooley, Agrim Gupta, Ajay Mandekar, Ajinkya Jain, et al. Open x-embodiment: Robotic learning datasets and rt-x models: Open x-embodiment collaboration 0. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903. IEEE, 2024.
- A Paszke. Pytorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703*, 2019.
- Karl Pertsch, Kyle Stachowicz, Brian Ichter, Danny Driess, Suraj Nair, Quan Vuong, Oier Mees, Chelsea Finn, and Sergey Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021.
- Jathushan Rajasegaran, Ilija Radosavovic, Rahul Ravishankar, Yossi Gandelsman, Christoph Feichtenhofer, and Jitendra Malik. An empirical study of autoregressive pre-training from videos. *arXiv preprint arXiv:2501.05453*, 2025.
- C Schuhmann, A Köpf, R Vencu, T Coombes, and R Beaumont. Laion coco: 600m synthetic captions from laion2b-en. *URL https://laion.ai/blog/laion-coco*, 2022.
- Younggyo Seo, Danijar Hafner, Hao Liu, Fangchen Liu, Stephen James, Kimin Lee, and Pieter Abbeel. Masked world models for visual control. In *Conference on Robot Learning*, pages 1332–1344. PMLR, 2023.
- Mustafa Shukor and Matthieu Cord. Skipping computations in multimodal llms. *arXiv preprint arXiv:2410.09454*, 2024.

- Mustafa Shukor, Corentin Dancette, and Matthieu Cord. ep-alm: Efficient perceptual augmentation of language models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 22056–22069, 2023a.
- Mustafa Shukor, Corentin Dancette, Alexandre Rame, and Matthieu Cord. Unival: Unified model for image, video, audio and language tasks. *Transactions on Machine Learning Research Journal*, 2023b.
- Mustafa Shukor, Enrico Fini, Victor Guilherme Turrissi da Costa, Matthieu Cord, Joshua Susskind, and Alaaeldin El-Nouby. Scaling laws for native multimodal models. *arXiv preprint arXiv:2504.07951*, 2025.
- Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. In C. Cortes, N. Lawrence, D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems*, volume 28. Curran Associates, Inc., 2015. https://proceedings.neurips.cc/paper_files/paper/2015/file/8d55a249e6baa5c06772297520da2051-Paper.pdf.
- Shengkun Tang, Yaqing Wang, Zhenglun Kong, Tianchi Zhang, Yao Li, Caiwen Ding, Yanzhi Wang, Yi Liang, and Dongkuan Xu. You need multiple exiting: Dynamic early exiting for accelerating unified vision language model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10781–10791, 2023.
- Chameleon Team. Chameleon: Mixed-modal early-fusion foundation models. *arXiv preprint arXiv:2405.09818*, 2024.
- Gemini Team, Rohan Anil, Sebastian Borgeaud, Yonghui Wu, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- Octo Model Team, Dibya Ghosh, Homer Walke, Karl Pertsch, Kevin Black, Oier Mees, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, et al. Octo: An open-source generalist robot policy. *arXiv preprint arXiv:2405.12213*, 2024.
- Peter Tong, Ellis Brown, Penghao Wu, Sanghyun Woo, Adithya Jairam Vedagiri IYER, Sai Charitha Akula, Shusheng Yang, Jihan Yang, Manoj Middepogu, Ziteng Wang, et al. Cambrian-1: A fully open, vision-centric exploration of multimodal llms. *Advances in Neural Information Processing Systems*, 37:87310–87356, 2024.
- Maria Tsimpoukelli, Jacob L Menick, Serkan Cabi, SM Eslami, Oriol Vinyals, and Felix Hill. Multimodal few-shot learning with frozen language models. *Advances in Neural Information Processing Systems*, 34:200–212, 2021.
- Th  ophane Valla  ys, Mustafa Shukor, Matthieu Cord, and Jakob Verbeek. Improved baselines for data-efficient perceptual augmentation of llms. *arXiv preprint arXiv:2403.13499*, 2024.
- A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.
- Peng Wang, An Yang, Rui Men, Junyang Lin, Shuai Bai, Zhikang Li, Jianxin Ma, Chang Zhou, Jingren Zhou, and Hongxia Yang. Ofa: Unifying architectures, tasks, and modalities through a simple sequence-to-sequence learning framework. In *International conference on machine learning*, pages 23318–23340. PMLR, 2022.
- Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model’s perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.
- Yi Wang, Xinhao Li, Ziang Yan, Yinan He, Jiashuo Yu, Xiangyu Zeng, Chenting Wang, Changlian Ma, Haian Huang, Jianfei Gao, et al. Internvideo2. 5: Empowering video mllms with long and rich context modeling. *arXiv preprint arXiv:2501.12386*, 2025.
- Junjie Wen, Yichen Zhu, Jinming Li, Minjie Zhu, Kun Wu, Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, Yaxin Peng, et al. Tinyvla: Towards fast, data-efficient vision-language-action models for robotic manipulation. *arXiv preprint arXiv:2409.12514*, 2024.
- Junjie Wen, Yichen Zhu, Jinming Li, Zhibin Tang, Chaomin Shen, and Feifei Feng. Dexvla: Vision-language model with plug-in diffusion expert for general robot control. *arXiv preprint arXiv:2502.05855*, 2025.
- Annie Xie, Lisa Lee, Ted Xiao, and Chelsea Finn. Decomposing the generalization gap in imitation learning for visual robotic manipulation. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 3153–3160. IEEE, 2024.
- Yuan Yao, Tianyu Yu, Ao Zhang, Chongyi Wang, Junbo Cui, Hongji Zhu, Tianchi Cai, Haoyu Li, Weilin Zhao, Zhihui He, Qianyu Chen, Huarong Zhou, Zhensheng Zou, Haoye Zhang, Shengding Hu, Zhi Zheng, Jie Zhou, Jie Cai, Xu Han, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. Minicpm-v: A gpt-4v level mllm on your phone, 2024. <https://arxiv.org/abs/2408.01800>.
- Tianhe Yu, Deirdre Quillen, Zhanpeng He, Ryan Julian, Karol Hausman, Chelsea Finn, and Sergey Levine. Meta-world: A benchmark and evaluation for multi-task and meta reinforcement learning. In *Conference on robot learning*, pages 1094–1100. PMLR, 2020.

- Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11975–11986, 2023.
- Boqiang Zhang, Kehan Li, Zesen Cheng, Zhiqiang Hu, Yuqian Yuan, Guanzheng Chen, Sicong Leng, Yuming Jiang, Hang Zhang, Xin Li, et al. Videollama 3: Frontier multimodal foundation models for image and video understanding. *arXiv preprint arXiv:2501.13106*, 2025.
- Tony Z Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- Baichuan Zhou, Ying Hu, Xi Weng, Junlong Jia, Jie Luo, Xien Liu, Ji Wu, and Lei Huang. Tinyllava: A framework of small-scale large multimodal models. *arXiv preprint arXiv:2402.14289*, 2024.
- Deyao Zhu, Jun Chen, Xiaoqian Shen, Xiang Li, and Mohamed Elhoseiny. MiniGPT-4: Enhancing vision-language understanding with advanced large language models. In *The Twelfth International Conference on Learning Representations*, 2024. <https://openreview.net/forum?id=1tZbq88f27>.
- Wanrong Zhu, Jack Hessel, Anas Awadalla, Samir Yitzhak Gadre, Jesse Dodge, Alex Fang, Youngjae Yu, Ludwig Schmidt, William Yang Wang, and Yejin Choi. Multimodal c4: An open, billion-scale corpus of images interleaved with text. In *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. <https://openreview.net/forum?id=tOd8rSjcWz>.

A Appendix

A.1 Community datasets

Task annotation For task annotation, we prompt the VLM with the following:

Here is a current task description: {current_task}. Generate a very short, clear, and complete one-sentence describing the action performed by the robot arm (max 30 characters). Do not include unnecessary words. Be concise.

Here is some examples: Pick up the cube and place it in the box, open the drawer and so on. Start directly with an action verb like “Pick”, “Place”, “Open”, etc.

Similar to the provided examples, what is the main action done by the robot arm?

List of datasets.

satvikahuja/mixer_on_off_new_1, aergogo/so100_pick_place, andy309/so100_0314_fold_cloths
jchun/so100_pickplace_small_20250323_120056, astroyat/cube, Ofiroz91/so_100_cube2bowl
HappyPablo/dec3_data2, ZCM5115/so100_1210, francescocrivelli/orange_feeding
francescocrivelli/carrot_eating, 0x00raghu/toffee_red, 0x00raghu/toffee_red_2
0x00raghu/toffee_red_3_, 0x00raghu/toffee_blue, 0x00raghu/toffee_blue_2
0x00raghu/toffee_to_hand_1, 0x00raghu/toffee_to_hand_2, liyitenga/so100_bi_hello
liyitenga/so100_bi_giveme5, ZCM5115/so100_2Arm3cameras_movebox, pranavsaroha/so100_carrot_1
pranavsaroha/so100_carrot_3, pranavsaroha/so100_carrot_4, maximilienroberti/so100_lego_red_box
pranavsaroha/so100_squishy, rabhishek100/so100_train_dataset, pranavsaroha/so100_squishy100
swarajgosavi/kikobot_pusht_real_v2, pandaRQ/pickmed, swarajgosavi/act_kikobot_pusht_real
pranavsaroha/so100_squishy2colors, pranavsaroha/so100_squishy2colors_1, Chojins/chess_game_001_white
jmrog/so100_sweet_pick, Chojins/chess_game_002_white, pranavsaroha/so100_squishy2colors_2_new
Chojins/chess_game_003_white, aractingi/pick_place_lego_cube, Chojins/chess_game_004_white
Chojins/chess_game_005_white, Chojins/chess_game_006_white, Chojins/chess_game_007_white
koenvanwijk/blue2, jlitch/so100multicam3, koenvanwijk/blue52
jlitch/so100multicam6, aractingi/pick_place_lego_cube_1, jlitch/so100multicam7
vladfatu/so100_ds, Chojins/chess_game_000_white, HITHY/so100-kiwi
HITHY/so100_peach1, HITHY/so100_redstrawberry, satvikahuja/orange_mixer_1
satvikahuja/mixer_on_off, satvikahuja/orange_pick_place_new1, satvikahuja/mixer_on_off_new
danmac1/real_real332, FeiYjf/Makalu_push, liyitenga/so100_pick_taffy1
chmadran/so100_dataset04, FeiYjf/Maklu_dataset, FeiYjf/new_Dataset
liyitenga/so100_pick_taffy2, satvikahuja/mixer_on_off_new_4, CSCSXX/pick_place_cube_1.17
liyitenga/so100_pick_taffy3, liyitenga/so100_pick_taffy4, yuz1wan/so100_pick_pink
yuz1wan/so100_pick_wahaha, yuz1wan/so100_pp_pink, yuz1wan/so100_pour_cup

liyitenga/so100_pick_taffy5, liyitenga/so100_pick_taffy6, yuz1wan/so100_button
 yuz1wan/so100_pickplace, liyitenga/so100_pick_taffy7, FeiYjf/push_gg
 FeiYjf/push_0094, swarajgosavi/act_kikobot_block_real, liyitenga/so100_pick_taffy8
 phospho-ai/OrangeBrick3Cameras, vaishanthr/toy_pick_place, SeanLMH/so100_picknplace_v2
 pepijn223/yellow_lego_in_box1, DimiSch/so100_50ep_2, DimiSch/so100_50ep_3
 SeanLMH/so100_picknplace, nbaron99/so100_pick_and_place2, chmadran/so100_dataset08
 vaishanthr/toy_pickplace_50ep, Beegbrain/pick_place_green_block_lr, Ityl/so100_recording1
 vaishanthr/toy_pickplace, ad330/so100_box_pickPlace, Beegbrain/so100_put_cube_cup
 aractingi/push_green_cube_hf, aractingi/push_green_cube_hf_cropped_resized,
 carpit680/giraffe_task, carpit680/giraffe_sock_demo_1, DimiSch/so100_terra_50_2,
 carpit680/giraffe_sock_demo_2, aractingi/push_cube_to_face_reward
 aractingi/push_cube_to_face_reward_cropped_resized, aractingi/push_cube_reward_data
 aractingi/push_cube_reward_data_cropped_resized, aractingi/push_cube_offline_data_cropped_resized,
 aractingi/push_cube_front_side_reward, aractingi/push_cube_front_side_reward_cropped_resized,
 aractingi/push_cube_front_side_reward_long, aractingi/push_cube_front_side_reward_long_cropped_resized
 aractingi/push_cube_reward, aractingi/push_cube_reward_cropped_resized,
 aractingi/push_cube_square_reward_cropped_resized, aractingi/push_cube_square_reward_1,
 aractingi/push_cube_square_reward_1_cropped_resized, aractingi/push_cube_square_light_reward
 aractingi/push_cube_square_light_offline_demo
 aractingi/push_cube_square_light_offline_demo_cropped_resized
 denghj/dataset_red_tape01, aractingi/push_cube_square_offline_demo,
 aractingi/push_cube_square_offline_demo_cropped_resized, Beegbrain/stack_two_cubes, FeiYjf/Test_NNNN
 LegrandFrederic/Orange-brick-lower-resolution, aractingi/pick_place_lego_cube_cropped_resized
 aractingi/push_cube_overfit, aractingi/push_cube_overfit_cropped_resized, HITHY/so100_peach
 zaringleb/so100_cube_2, andreasBihlmaier/dual_arm_transfer_2025_02_16, zaringleb/so100_cube_4_binary
 1g0rrr/reward_pickplace1, 1g0rrr/reward_pickplace1_cropped_resized, FeiYjf/Hold_Pieces
 FeiYjf/Grab_Pieces, hegdearyandev/so100_eraser_cup_v1, jbraumann/so100_1902
 liyitenga/so100_pick_taffy10, mikechambers/block_cup_5, zaringleb/so100_cube_5_linear
 yuz1wan/so100_pickplace_0223_2, yuz1wan/so100_pickplace_0223_3, samsam0510/mj_data_temp
 samsam0510/tape_insert_1, samsam0510/tape_insert_2, pengjunkun/so100_push_to_hole
 Deason11/Random_Kitchen, 1g0rrr/reward_dataset_name2, 1g0rrr/reward_dataset_name2_cropped_resized
 1g0rrr/offline_dataset_name2, 1g0rrr/offline_dataset_name2_cropped_resized
 aractingi/push_cube_simp_cropped_resized, danielkr452/so100_work6
 Loki0929/so100_100, yuz1wan/so100_fold_0227_1
 yuz1wan/so100_fold_0227_2, speedyoshi/so100_grasp_pink_block, lirislab/stack_two_red_cubes
 lirislab/red_cube_into_mug, lirislab/green_lego_block_into_mug, lirislab/green_lego_block_into_mug_easy
 kevin510/lerobot-cat-toy-placement, NONHUMAN-RESEARCH/SOARM100_TASK_VENDA_BOX, wangjl1512/pour_water
 airthebear/so100_GL, zijian2022/noticehuman1, zijian2022/noticehuman2
 kantine/so100_kapla_tower6, zijian2022/noticehuman5, zijian2022/llm40
 Ashton3/lerobot-aloha, zijian2022/noticehuman50, AaronNewman/screwdriver_task_batch1
 AaronNewman/screwdriver_task_batch2, AaronNewman/screwdriver_task_batch3, zijian2022/noticehuman60
 zijian2022/noticehuman70, Bartm3/tape_to_bin, liuhuanjim013/so100_th_1
 Pi-robot/barbecue_flip, Pi-robot/barbecue_put, wangjl1512/doll
 sshh11/so100_orange_50ep_1, sshh11/so100_orange_50ep_2, DorayakiLin/so100_pick_cube_in_box
 Bartm3/tape_to_bin2, luke250305/play_dice_250311.1, andy309/so100_0311_1152
 sihyun77/suho_so100, sihyun77/si_so100, shreyasgite/so100_base_left
 sihyun77/suho_red, liuhuanjim013/so100_block, andy309/so100_0313_no_wrist_camera
 zijian2022/l9, zijian2022/n1_2, DorayakiLin/so100_stack_cube
 andy309/so100_0313_no_wrist_camera_with_two_arms_cloths, joaoocruz00/so100_makeitD1, zijian2022/l10_1
 zijian2022/l10_5, sihyun77/suho_red2, sihyun77/suho_angel
 sihyun77/sihyun_king, acrampette/third_arm_01, Winster/so100_cube
 1g0rrr/sam_openpi03, thedevansh/mar16_1336, hkphooey/throw_stuffie
 doujiangwang/task1_10epi_100000step, sihyun77/sihyun_3_17_1, acrampette/third_arm_02
 imsyed00/so100_yellowbowl_pickplace_1, kumarhans/so100_tape_task, sihyun77/sihyun_main
 doujiangwang/task2_10epi_100000step, kantine/industrial_robothon_buttons_expert
 kantine/industrial_robothon_buttons_anomaly, kantine/industrial_robothon_hatchAndProbe_expert
 kantine/industrial_robothon_hatchAndProbe_anomaly, Odog16/so100_tea_towel_folding_v1

zijian2022/so100_318, zijian2022/so100_318_1,
Congying1112/so100_place_blue_bottle_with_two_cameras
Congying1112/so100_place_blue_bottle_with_two_cameras2,
Congying1112/so100_place_blue_bottle_with_single_camera, pietroom/first_task_short
kantine/industrial_screws_sorting_expert, kantine/industrial_screws_sorting_anomaly, pietroom/second_task
zijian2022/c0, doujiangwang/task4_10epi_100000step, Congying1112/so100_switch_with_onhand_camera
HYAIYN/so100_get_orange_10epi, doujiangwang/task5_10epi_100000step, 1g0rrr/sam_openpi_cube_low10
1g0rrr/sam_openpi_cube_top10, 1g0rrr/sam_openpi_wire10, 1g0rrr/sam_openpi_solder1
1g0rrr/sam_openpi_solder2, wcode/so100_put_pen_50, jchun/so100_pickplace_small_20250322_193929
bnarin/so100_tic_tac_toe_we_do_it_live, dc2ac/so100-t5, chmadran/so100_home_dataset
baladhurgesh97/so100_final_picking_3, bnarin/so100_tic_tac_toe_move_0_0
bnarin/so100_tic_tac_toe_move_1_0, bnarin/so100_tic_tac_toe_move_2_1
bnarin/so100_tic_tac_toe_move_4_0, zaringleb/so100_cube_6_2d
andlyu/so100_indoor_0, andlyu/so100_indoor_2, Winster/so100_sim
badwolf256/so100_twin_cam_duck, Congying1112/so100_simplepick_with_2_cameras_from_top,
andlyu/so100_indoor_4, Zak-Y/so100_grap_dataset
kantine/domotic_pouringCoffee_expert, kantine/domotic_pouringCoffee_anomaly
lucasngoo/so100_strawberry_grape, kantine/domotic_makingCoffee_expert
kantine/domotic_makingCoffee_anomaly, ZGGZZG/so100_drop1
kantine/industrial_soldering_expert, kantine/industrial_soldering_anomaly
Yotofu/so100_sweeper_shoes, kantine/domotic_dishTidyUp_expert
kantine/domotic_dishTidyUp_anomaly, kantine/domotic_groceriesSorting_expert
kantine/domotic_groceriesSorting_anomaly, badwolf256/so100_twin_cam_duck_v2
kantine/domotic_vegetablesAndFruitsSorting_expert
kantine/domotic_vegetablesAndFruitsSorting_anomaly, kantine/domotic_setTheTable_expert
kantine/domotic_setTheTable_anomaly, therarelab/so100_pick_place, abhisb/so100_51_ep
andlyu/so100_indoor_val_0, lizi178119985/so100_jia, badwolf256/so100_twin_cam_duck_v3
andrewcole712/so100_tape_bin_place, Gano007/so100_lolo, Zak-Y/so100_three_cameras_dataset
Gano007/so100_doliprane, XXRRSSRR/so100_v3_num_episodes_50, zijian2022/assemblyarm2
ganker5/so100_action_20250403, andlyu/so100_indoor_val2, Gano007/so100_gano
paszea/so100_whale_grab, paszea/so100_whale, Clementppr/lerobot_pick_and_place_dataset_world_model
andlyu/so100_indoor_10, RasmusP/so100_dataset50ep_a, RasmusP/so100_dataset50ep
Gano007/so100_second, zaringleb/so100_cude_linear_and_2d_comb, dsfsg/grasp_pens
zijian2022/digitalfix, zijian2022/digitalfix2, zijian2022/digitalfix3
T1g3rGE/so100_pickplace_small_20250407_171912, sihyun77/mond_13
abokinala/sputnik_100_11_pick_place_container, dsfsg/bring_bottle
abokinala/sputnik_100_12_pick_place_container, Mwuqiu/so100_0408
AK51/4090_01, 356c/so100_rope_reposition_1, paszea/so100_lego_mix
abokinala/sputnik_100_14_pick_place_container, abokinala/sputnik_100_23_pick_place_surface,
jiajun001/eraser00_2, jlesein/TestBoulon2
duthvik/sputnik_100_31_pour_liquid, duthvik/sputnik_100_24_pick_place_surface
duthvik/sputnik_100_25_pick_place_surface, duthvik/sputnik_100_17_pick_place_container
duthvik/sputnik_100_26_pick_place_surface, VoicAndrei/so100_banana_to_plate_rebel_full
isadev/bougies1, danaaubakirova/so100_task_1
danaaubakirova/so100_task_2, danaaubakirova/so100_task_3, danaaubakirova/so100_task_4
sixpigs1/so100_pick_cube_in_box_error, sixpigs1/so100_push_cube_error, sixpigs1/so100_pull_cube_error
isadev/bougies2, therarelab/med_dis_rare_6, duthvik/sputnik_100_27_pick_place_surface
zijian2022/closer3, duthvik/sputnik_100_41_custom_tasks, duthvik/sputnik_100_42_custom_tasks
duthvik/sputnik_100_43_custom_tasks, duthvik/sputnik_100_44_custom_tasks,
duthvik/sputnik_100_51_kitchen_tasks, duthvik/sputnik_100_52_kitchen_tasks
duthvik/sputnik_100_53_kitchen_tasks, duthvik/sputnik_100_45_custom_tasks
duthvik/sputnik_100_32_pour_liquid, duthvik/sputnik_100_29_pick_place_surface
duthvik/sputnik_100_18_pick_place_container
sixpigs1/so100_pull_cube_by_tool_error, sixpigs1/so100_insert_cylinder_error
abokinala/sputnik_100_54_kitchen_tasks, abokinala/sputnik_100_55_kitchen_tasks, m1b/so100_bluelego
abokinala/sputnik_100_46_custom_tasks
m1b/so100_bluelego_updt, kantine/flip_A0, kantine/flip_A1

kantine/flip_A2, kantine/flip_A3, lirislab/guess_who_no_cond
 kantine/flip_A4, kantine/flip_A5, lirislab/guess_who_lighting
 nguyen-v/so100_press_red_button, nguyen-v/so100_bimanual_grab_lemon_put_in_box2, pierfabre/cow
 nguyen-v/press_red_button_new, nguyen-v/so100_rotate_red_button, CidoYi/so100_all_notes
 roboticshack/team10-red-block, CidoYi/so100_all_notes_1, roboticshack/team_5-QuiEstCe_everyBox
 roboticshack/team11_pianobot, roboticshack/team2-guess_who_so100
 roboticshack/team2-guess_who_so100_light, roboticshack/team2-guess_who_so100_edge_case
 roboticshack/team2-guess_who_less_ligth, CidoYi/so100_all_notes_3
 dsfsg/grasp_pen_and_bottle, abokinala/sputnik_100_60_kitchen_tasks
 abokinala/sputnik_100_58_kitchen_tasks, danaaubakirova/so100_v2_task_1
 danaaubakirova/so100_v2_task_2, danaaubakirova/so100_v2_task_3
 danaaubakirova/so100_v2_task_4, zijian2022/force1, zijian2022/force2
 zijian2022/force3, jiajun001/eraser00_3, zijian2022/bi2
 zijian2022/bi1, zijian2022/hand1, Setchii/so100_grab_ball
 MossProphet/so100_square-1-2-3.2, pierfabre/rabbit
 bensprenger/right_arm_p_brick_in_box_with_y_noise_v0
 pierfabre/horse, pierfabre/pig2, pierfabre/pig3
 pierfabre/cow2, pierfabre/sheep, Chojins/chess_game_009_white
 sihyun77/suho_3_17_1, sihyun77/sihyun_3_17_2, sihyun77/suho_3_17_3
 sihyun77/sihyun_3_17_5, Odog16/so100_cube_drop_pick_v1, sihyun77/sihyun_main_2
 sihyun77/suho_main_2, Bartm3/dice2, sihyun77/sihyun_main_3
 Loki0929/so100_duck, pietroom/holdthis, pietroom/actualeasytask
 Beegbrain/pick_lemon_and_drop_in_bowl, Beegbrain/sweep_tissue_cube, zijian2022/321
 gxy1111/so100_pick_place, Odog16/so100_cube_stacking_v1, sihyun77/mond_1
 andlyu/so100_indoor_1, andlyu/so100_indoor_3, frk2/so100large
 lirislab/sweep_tissue_cube, lirislab/lemon_into_bowl, lirislab/red_cube_into_green_lego_block
 lirislab/red_cube_into_blue_cube, 00ri/so100_battery, frk2/so100largediffcam
 FsqZ/so100_1, ZGGZZG/so100_drop0, Chojins/chess_game_000_white_red
 smanni/train_so100_fluffy_box, ganker5/so100_push_20250328, ganker5/so100_dateline_0328
 ganker5/so100_color_0328, CrazyYhang/A1234-B-C_mvA2B, RasmusP/so100_Orange2Green
 sixpigs1/so100_pick_cube_in_box, ganker5/so100_push_20250331, ganker5/so100_dateline_20250331
 lirislab/put_caps_into_teabox, lirislab/close_top_drawer_teabox, lirislab/open_top_drawer_teabox
 lirislab/unfold_bottom_right, lirislab/push_cup_target, lirislab/put_banana_bowl
 Chojins/chess_game_001_blue_stereo, Chojins/chess_game_001_red_stereo, ganker5/so100_toy_20250402
 Gano007/so100_medic, 00ri/so100_battery_bin_center, paszea/so100_whale_2
 lirislab/fold_bottom_right, lirislab/put_coffee_cap_teabox, therarelab/so100_pick_place_2
 paszea/so100_whale_3, paszea/so100_whale_4, paszea/so100_lego
 LemonadeDai/so100_coca, zijian2022/backgrounda, zijian2022/backgroundb
 356c/so100_nut_sort_1, Mwuiqu/so100_0408_muti, aimihat/so100_tape
 lirislab/so100_demo, 356c/so100_duck_reposition_1, zijian2022/sort1
 weiyel1/so100_410_zwy, VoicAndrei/so100_banana_to_plate_only, sixpigs1/so100_stack_cube_error
 isadev/bougies3, zijian2022/close3, bensprenger/left_arm_yellow_brick_in_box_v0
 lirislab/guess_who_so100, bensprenger/left_arm_yellow_brick_in_box_with_purple_noise_v0,
 roboticshack/team16-can-stacking, zijian2022/insert2
 roboticshack/team-7-right-arm-grasp-tape, Jiangeng/so100_413
 roboticshack/team9-pick_cube_place_static_plate
 Andrej0rsula/lerobot_double_ball_stacking_random
 roboticshack/left-arm-grasp-lego-brick
 roboticshack/team-7-left-arm-grasp-motor, roboticshack/team9-pick_chicken_place_plate
 roboticshack/team13-two-balls-stacking, tkc79/so100_lego_box_1
 roboticshack/team13-three-balls-stacking, pierfabre/chicken
 roboticshack/team16-water-pouring, ad330/cubePlace, Jiafei1224/so100_pa222per
 paszea/so100_lego_2cam, bensprenger/chess_game_001_blue_stereo, Mohamedal/put_banana
 tkc79/so100_lego_box_2, samanthalhy/so100_herding_1, jlesein/TestBoulon7
 pranavsaroha/so100_onelego2, pranavsaroha/so100_onelego3, pranavsaroha/so100_carrot_2
 vladfatu/so100_above, koenvanwijk/orange50-1, CSCSXX/pick_place_cube_1.18
 dragon-95/so100_sorting, dragon-95/so100_sorting_1, nbaron99/so100_pick_and_place4

Beegbrain/pick_place_green_block, dragon-95/so100_sorting_3, HITHY/so100_peach3
shreyasgite/so100_legocube_50, triton7777/so100_dataset_mix, NONHUMAN-RESEARCH/SOARM100_TASK_VENDA
mikechambers/block_cup_14, samsam0510/tooth_extraction_3, samsam0510/tooth_extraction_4
samsam0510/cube_reorientation_2, samsam0510/cube_reorientation_4, samsam0510/glove_reorientation_1
vladfatu/so100_office, pranavsarooha/so100_legos4, Ityl/so100_recording2
FeiYjf/new_GtoR, dragon-95/so100_sorting_2, HITHY/so100_peach4
jpata/so100_pick_place_tangerine, HITHY/so100_strawberry, shreyasgite/so100_base_env
koenvanwijk/orange50-variation-2, pranavsarooha/so100_carrot_5, pandaRQ/pick_med_1
aractingi/push_cube_offline_data, DorayakiLin/so100_pick_charger_on_tissue, zijian2022/noticehuman3
liuhuanjim013/so100_th